

Plan de Pruebas Avances 2



David Alexander Santisteban Apolinar

Andrés Julián Mendivelso Chaparro

Daniel Alejandro Rojas Saavedra

Samuel Andres Bustos Mosquera

Facultad de Ingeniería, Universidad Libre

Ingeniería de Software III

Tabla de Contenido

REPOSITORIO.....	5
PRIMER CORTE.....	6
1. Nombre del proyecto.....	6
2. Nombre del producto.....	6
3. Logo y slogan del producto.....	7
4. Tabla resumen de los requerimientos Funcionales y no Funcionales.....	7
4.1 Requerimientos Funcionales.....	8
4.2 Requerimientos No Funcionales.....	13
5. Historias de Usuario.....	14
5.1 Criterios de Aceptación Clave.....	18
6. Objetivos del plan de pruebas.....	20
6.1 Contexto del Proyecto.....	20
6.2 Framework y Tecnología Principal.....	20
6.3 Estructura MVC del Proyecto.....	21
6.3.1 MODELO — La capa de datos.....	21
6.3.2 CONTROLADOR — La capa de lógica de negocio.....	22
6.3.3 VISTA — La capa de interfaz de usuario.....	23
6.3.4 RUTAS API — El puente entre Vista y Controlador.....	25
6.4 Roles de Usuario.....	25
6.5 Niveles de Prueba y su Relación con el MVC.....	26
6.6 Objetivos de Pruebas Unitarias.....	27

	3
6.7 Objetivos de Pruebas de Integración.....	31
6.8 Objetivos de Pruebas de Sistema.....	34
6.9 Objetivos de Pruebas de Aceptación.....	38
6.10 Resumen de Objetivos y Trazabilidad.....	42
7. Clases y tipos de pruebas a realizar.....	43
7.1 Clases y Tipos de Prueba.....	43
7.1.1 Clases de Prueba.....	43
7.1.2 Tipos de Prueba.....	44
7.2 Aplicación por Nivel de Prueba.....	45
8. Herramientas a utilizar para las pruebas.....	46
8.1 Herramientas de Prueba.....	46
8.2 Resumen de Herramientas por Nivel.....	46
8.3 Descripción de cada Herramienta.....	47
AVANCES SEGUNDO CORTE.....	49
9. Implementación de Pruebas.....	49
9.1 Configuración del entorno de pruebas.....	50
9.1.1 Definición pytest.....	50
9.1.2 Archivo conftest.py como núcleo de configuración.....	51
9.1.3 Uso de fixtures en la configuración de pruebas.....	52
9.1.4 Estructura de las principales fixtures.....	53
9.1.4.1 Base de datos en memoria.....	53
9.1.4.2 Usuarios de prueba.....	55
9.1.4.3 Sesiones simuladas.....	57

	4
9.1.4.4 Datos del dominio.....	58
9.1.5 Organización del conjunto de pruebas.....	59
9.1.6 Tabla de trazabilidad.....	60
9.2 Implementación Pruebas Unitarias.....	64
9.2.1 Definición pruebas unitarias.....	64
9.2.2 Justificación del uso de caja blanca.....	64
9.2.3 Fixtures Utilizadas en las Pruebas Unitarias.....	65
9.2.4 Principio de aislamiento por test.....	71
9.2.5 Análisis de Clases de Prueba.....	72
9.2.5.1 Clase: TestAutenticacion.....	73
9.2.5.2 Clase: TestRegistrarVenta.....	75
9.2.5.3 Clase: TestAnularVenta.....	77
9.2.5.4 Clase: TestMovimientosInventario.....	78
9.2.5.5 Clase: TestAlertasStock.....	79
9.2.5.6 Clase: TestBalanceEconomico.....	81
9.2.5 Rol de los controladores.....	82
9.2.6 Casos de Frontera Documentados.....	82
9.2.7 Resultados de Ejecución.....	83
9.3 Implementación Pruebas de Integración.....	85
9.3.1 Definición de prueba de integración.....	85
9.3.2 Justificación del uso de caja gris.....	86
9.3.3 Análisis de Cada Clase de Prueba.....	88
9.3.3.1 TestAuditoriaDeAcceso.....	88

9.3.3.2 TestAtomicidadDeVentas.....	90
9.3.3.3 TestAnulacionRestaurandoStock.....	92
9.3.3.4 TestAlertasTrasMovimiento.....	94
9.3.3.5 TestControlDeAccesoHTTP.....	96
9.3.4 Casos de Frontera Documentados.....	99
9.3.5 Resultados de Ejecución.....	100
9.4 Implementación Pruebas de Sistema.....	100
9.4.1 Definición de Prueba de Sistema.....	100
9.4. 2 Justificación del Uso de Caja Negra.....	101
9.4.3 Análisis Detallado de Cada Clase de Prueba.....	103
9.4.3.1 TestFlujoDeVenta.....	103
9.4.3.2 TestAnulacionDeVentas.....	106
9.4.3.3 TestAlertasDeInventario.....	108
9.4.3.4 TestControlDeRoles.....	110
9.4.4 Casos de Frontera Documentados.....	113
9.4.5 Resultados de Ejecución.....	114
9.5 Conclusión General de Pruebas.....	115

REPOSITORIO

Link al repositorio de github:

https://github.com/DavidSantisteban/ING_SOFTWARE_III

PRIMER CORTE

1. Nombre del proyecto

El proyecto llevará el nombre de StoreVision, y tendrá como punto de partida el Autoservicio Don Julián, negocio que inspiró y motivó el desarrollo de esta solución. Si bien el sistema nacerá atendiendo las necesidades concretas de este establecimiento, la visión a futuro es que StoreVision pueda expandirse y ser adoptado por otras tiendas pequeñas y de mediano alcance en Colombia. La idea es construir desde un caso real una herramienta que entienda cómo funcionan estos negocios y que, con el tiempo, se convierta en una opción accesible para cualquier tendero o comerciante que quiera modernizar la forma en que gestiona su negocio.

2. Nombre del producto

El producto se llamará StoreVision – Sistema de Gestión para Tiendas Colombianas, en su versión 1.0.0. Será una aplicación web que en su primera etapa estará orientada al Autoservicio Don Julián, pero que ha sido pensada y diseñada desde el inicio con la flexibilidad suficiente para adaptarse a otros negocios similares. Contará con módulos para registrar ventas, controlar el inventario, generar reportes y gestionar el acceso de los usuarios. Al estar construido sobre una base escalable, se espera que en el futuro pueda implementarse en distintos tipos de tiendas sin necesidad de grandes modificaciones, manteniendo siempre la facilidad de uso como prioridad.

3. Logo y slogan del producto.



un ícono que representa de forma directa y reconocible el concepto de tienda.

Aunque hoy ese ícono se asocia al Autoservicio Don Julián como cliente inicial, está pensado para que cualquier pequeño o mediano comercio colombiano pueda identificarse con él.

El eslogan que identificará al producto será:

"Sistema de gestión para tiendas colombianas"

Esta frase resume bien la intención del proyecto: no es una solución genérica, sino una herramienta pensada para el comercio colombiano, que nace desde la experiencia del Autoservicio Don Julián y que aspira a crecer junto a los negocios que más lo necesitan.

4. Tabla resumen de los requerimientos Funcionales y no Funcionales.

Para el desarrollo de StoreVision se identificaron ocho bloques de requerimientos funcionales con un total de 26 sub-requerimientos, y once requerimientos no funcionales, todos levantados a partir de las necesidades reales del Autoservicio Don Julián.

El documento con la documentación completa de requerimiento Funcionales y no Funcionales está presente en el siguiente enlace: [Requerimientos Funcionales y No funcionales StoreVision.docx](#)

A continuación, se presenta el resumen consolidado:

4.1 Requerimientos Funcionales

Código	Nombre	Descripción / Sub-requerimiento	Prioridad
RF1	Gestión de Ventas	Registrar, consolidar, consultar y anular transacciones de ventas en todas las sucursales	Alta
	RF1.1	Registrar automáticamente cada transacción de venta en caja con producto, cantidad, precio y hora en tiempo real	
	RF1.2	Consolidar automáticamente las ventas diarias por sucursal al final de la jornada	
	RF1.3	Permitir la consulta de ventas filtrando por día, semana, mes o rango de fechas personalizado	
	RF1.4	Permitir anular o corregir una venta únicamente bajo rol autorizado, registrando un log de la modificación	
RF2	Gestión de Inventario	Controlar entradas y salidas de productos con actualizaciones automáticas y alertas de stock bajo	Alta
	RF2.1	Registrar entradas (compras a proveedores)	

		y salidas (ventas, ajustes) con actualización inmediata del stock	
	RF2.2	Actualizar automáticamente el nivel de inventario después de cada venta registrada en el sistema	
	RF2.3	Generar alertas automáticas cuando el inventario de un producto baje del mínimo establecido	
	RF2.4	Permitir consultar el historial de movimientos de inventario por producto, sucursal y periodo de tiempo	
RF3	Reportes y Análisis	Generar reportes visuales de ventas, inventarios, productos más vendidos y balances económicos	Alta
	RF3.1	Generar reportes visuales con gráficos y tablas de ventas, inventarios y rendimiento de productos	
	RF3.2	Mostrar indicadores automáticos de los productos más vendidos y de mayor rotación en el panel principal	
	RF3.3	Consolidar balances económicos por sucursal y generales con ingresos, costos, gastos y utilidades	
RF4	Roles de Usuario	Crear y gestionar usuarios con roles diferenciados: dueña, administradora y	Muy Alta

		auxiliares	
	RF4.1	Permitir la creación de usuarios con roles predefinidos: dueña, administradores de sucursal y auxiliares	
	RF4.2	Restringir funciones sensibles según el rol; solo la dueña puede acceder a balances globales	
RF5	Alertas y Notificaciones	Notificaciones automáticas por inventario bajo, caída en ventas y recordatorio de cierre de caja	Muy Alta
	RF5.1	Notificar automáticamente cuando el stock de un producto baje del umbral mínimo configurado	
	RF5.2	Notificar cuando las ventas caigan más del 20% respecto al periodo anterior	
	RF5.3	Enviar recordatorios automáticos a los administradores para realizar el cierre de caja diario	
RF6	Gestión de Compras y Gastos	Registrar compras a proveedores y gastos operativos con integración en reportes económicos	Alta
	RF6.1	Registrar compras a proveedores con detalle de producto, cantidad, costo y fecha, actualizando inventario y balance	

	RF6.2	Registrar gastos operativos (servicios, arriendo, transporte) y reflejarlos en reportes económicos mensuales	
	RF6.3	Integrar compras y gastos en reportes consolidados mostrando ingresos vs. egresos y margen de utilidad neta	
RF7	Exportación de Datos	Exportar reportes en PDF y Excel, y generar respaldos mensuales de la información	Baja
	RF7.1	Permitir exportar cualquier reporte generado (ventas, inventario, balances) en formatos PDF y Excel	
	RF7.2	Generar respaldo mensual consolidado de ventas, inventarios, compras y gastos de todas las sucursales	
RF8	Gestión de Auditoría y Trazabilidad	Registrar y consultar todas las acciones del sistema con filtros y reportes de auditoría	Alta
	RF8.1	Registrar automáticamente cada acción relevante: ventas anuladas, modificaciones de inventario, accesos fallidos	
	RF8.2	Permitir filtrar registros de auditoría por rango de fechas, usuario o tipo de acción, con exportación	

	RF8.3	Generar reportes mensuales de auditoría consolidados con las acciones críticas de todas las sucursales	
--	-------	--	--

4.2 Requerimientos No Funcionales

Código	Nombre	Descripción
RNF1	Rendimiento	Soportar mínimo 500 transacciones diarias por sucursal. Respuesta menor a 2 seg para ventas y menor a 5 seg para reportes
RNF2	Disponibilidad	Disponibilidad del 99% mensual con soporte offline parcial. Mantenimiento sin interrumpir registro de ventas
RNF3	Usabilidad	Interfaz intuitiva y responsive, aprendible en menos de 1 hora. Tareas frecuentes en máximo 3 pasos
RNF4	Escalabilidad	Soporte para nuevas sucursales y funcionalidades sin rediseño de arquitectura. Escalado horizontal disponible
RNF5	Mantenibilidad	Código modular y documentado siguiendo principios SOLID, con logs detallados para diagnóstico de errores
RNF6	Compatibilidad Técnica	Compatible con FastAPI, navegadores Chrome/Edge/Firefox. Comunicación mediante API RESTful con JSON
RNF7	Respaldo y Recuperación	Copias automáticas diarias. Recuperación en máximo 1 hora. Retención mínima de 30 días en entorno seguro

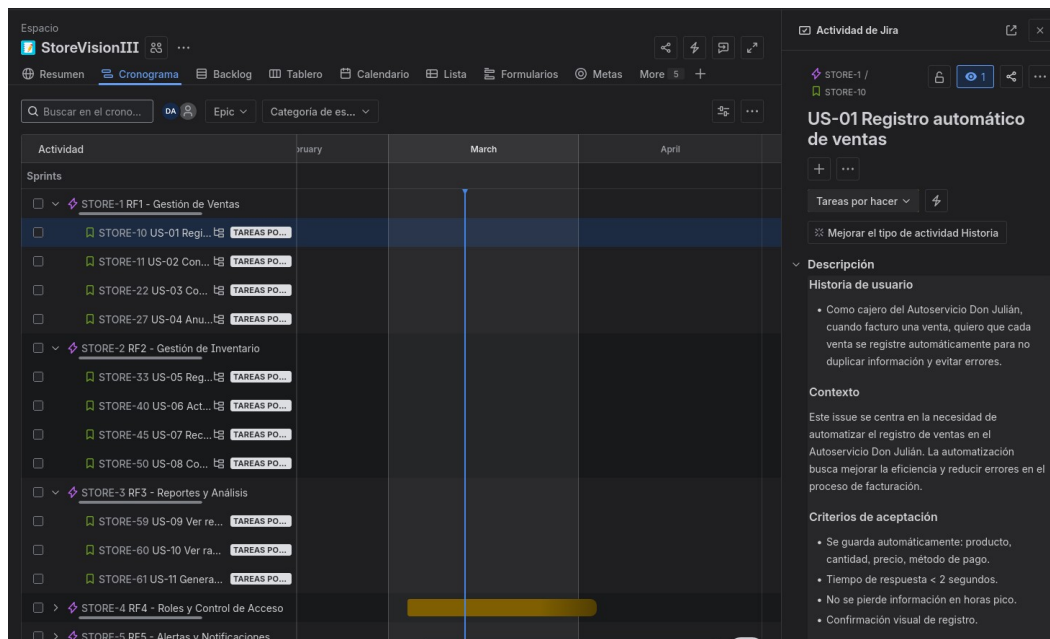
RNF8	Auditoría	Registro de todas las acciones con usuario, fecha y hora. Conservación mínima de 6 meses. Acceso solo para roles autorizados
RNF9	Confiabilidad	Tasa de error menor al 0.5%. Sin pérdida de datos ante fallos de red o energía. Sincronización con verificación de duplicados
RNF10	Interoperabilidad	Exportación en formatos estándar CSV, Excel y PDF. Datos con codificación UTF-8 y estándares de integración

En total, StoreVision contará con 8 requerimientos funcionales distribuidos en 26 sub-requerimientos, y 11 requerimientos no funcionales que garantizarán la calidad, seguridad y rendimiento del sistema una vez sea implementado en el Autoservicio Don Julián y en los negocios a los que se extienda en el futuro.

5. Historias de Usuario

Las historias de usuario del proyecto StoreVision están gestionadas en Jira bajo el proyecto STORE. El detalle completo de cada historia, los comentarios, el estado de avance y los archivos adjuntos se encuentran en el tablero correspondiente.

La siguiente imagen muestra un ejemplo la organización de las historias de usuario en Jira, en la imagen se muestran los 4 primeros requerimientos con sus correspondientes historias de usuario, a la derecha la descripción de los requerimientos.



También cada historia de usuario tiene sus propias Subtareas que ayudan a guiar sobre lo que se tiene que realizar en el proyecto para cumplir con los casos de usuario.

Subtareas				0 % completado	
Actividad	Pri...	Per...	Estado		
STORE-12 Optimizar el rendimiento del registro para respuesta < 2 segundos	= M	Si	TAREAS POR HACER		
STORE-13 Diseñar e implementar el registro automático de ventas al facturar	= M	Si	TAREAS POR HACER		
STORE-14 Realizar pruebas de estrés en horas pico	= M	Si	TAREAS POR HACER		
STORE-15 Validar integridad y no duplicidad de la información registrada	= M	Si	TAREAS POR HACER		
STORE-16 Implementar confirmación visual de registro exitoso	= M	Si	TAREAS POR HACER		

Link al Jira: [https://unilibre-team-wp8z723y.atlassian.net/jira/software/projects/STORE/boards/3/timeline?](https://unilibre-team-wp8z723y.atlassian.net/jira/software/projects/STORE/boards/3/timeline?atlOrigin=eyJpIjoiYjA1YjAyYWE4ZDQ5NGQxNDk4Y2RlMWJmOTc4ZmExMDIiLCJwIjoiaaiJ9)
[atlOrigin=eyJpIjoiYjA1YjAyYWE4ZDQ5NGQxNDk4Y2RlMWJmOTc4ZmExMDIiLCJwIjoiaaiJ9](https://unilibre-team-wp8z723y.atlassian.net/jira/software/projects/STORE/boards/3/timeline?atlOrigin=eyJpIjoiYjA1YjAyYWE4ZDQ5NGQxNDk4Y2RlMWJmOTc4ZmExMDIiLCJwIjoiaaiJ9)

Este documento presenta un resumen estructurado y la relación entre cada historia y los casos de prueba que la verifican.

La siguiente tabla muestra todas las historias organizadas por épica, indicando el rol del usuario que la protagoniza, la prioridad asignada en Jira y el módulo del sistema al que corresponde.

ID Jira	Historia	Rol	Prioridad	Módulo	Casos de prueba relacionados
EP-01 · Gestión de Ventas					
STORE-01	Registrar venta automáticamente	<i>Cajero</i>	Alta	Ventas	PU-08, PU-09, PU-10 PI-03, PI-04 PS-01, PS-02 PA-02, PA-03
STORE-02	Consolidar ventas diarias	<i>Administradora</i>	Alta	Ventas	PU-11 PI-05 PS-03 PA-04
STORE-03	Consultar ventas por periodo	<i>Administradora</i>	Media	Ventas	PU-12 PI-06 PS-04 PA-05
STORE-04	Anular o corregir una venta	<i>Administradora</i>	Alta	Ventas	PU-13 PI-07 PS-05, PS-06 PA-06
EP-02 · Gestión de Inventario					
STORE-05	Registrar entradas y salidas de inventario	<i>Administradora</i>	Alta	Inventario	PU-14, PU-15 PI-08 PS-07 PA-07
STORE-06	Actualizar inventario tras una venta	<i>Cajero</i>	Alta	Inventario	PU-16 PI-03 PS-01, PS-08 PA-02
STORE-07	Recibir alerta de stock mínimo	<i>Administradora</i>	Alta	Inventario	PU-17 PI-09 PS-09 PA-08
STORE-08	Consultar historial de movimientos	<i>Administradora</i>	Media	Inventario	PU-18 PI-10 PS-10 PA-09
EP-03 · Reportes y Análisis					
STORE-09	Ver reportes visuales de ventas e inventario	<i>Administradora</i>	Alta	Reportes	PU-19, PU-20 PI-11 PS-11 PA-10

STORE-1 0	Ver ranking de productos más vendidos	<i>Administradora</i>	Media	Reportes	PU-21 PI-11 PS-12 PA-11
STORE-1 1	Generar balance económico	<i>Dueña</i>	Alta	Reportes	PU-19 PI-11 PS-11 PA-10
EP-04 · Roles y Control de Acceso					
STORE-1 2	Crear usuarios con roles diferenciados	<i>Dueña</i>	Alta	Auth	PU-01, PU-06 PI-01, PI-02 PS-13 PA-12
STORE-1 3	Restringir acceso según rol	<i>Dueña</i>	Alta	Auth	PU-07 PI-12 PS-14, PS-15 PA-13, PA-14
EP-05 · Alertas y Notificaciones					
STORE-1 4	Notificar inventario bajo por app	<i>Administradora</i>	Alta	Inventario	PU-17 PI-09 PS-09 PA-08
STORE-1 5	Notificar caída significativa en ventas	<i>Dueña</i>	Media	Reportes	PU-20 PS-11 PA-10
STORE-1 6	Recordatorio de cierre de caja	<i>Admin. sucursal</i>	Media	Dashboard	PS-03 PA-04
EP-06 · Gestión de Compras y Gastos					
STORE-1 7	Registrar compra a proveedor	<i>Administradora</i>	Media	Inventario	PU-14 PI-08 PS-07 PA-07
STORE-1 8	Registrar gastos operativos	<i>Administradora</i>	Media	Reportes	PS-11 PA-10
STORE-1 9	Ver impacto de compras y gastos en reportes	<i>Dueña</i>	Media	Reportes	PU-19 PI-11 PS-11 PA-10
EP-07 · Exportación de Datos					
STORE-2 0	Exportar reportes en PDF y Excel	<i>Administradora</i>	Baja	Reportes	PS-12 PA-11

STORE-2 1	Generar respaldo mensual de información	<i>Dueña</i>	Baja	Sistema	PS-12 PA-11
EP-08 · Auditoría y Trazabilidad					
STORE-2 2	Registrar acciones críticas automáticamente	<i>Dueña</i>	Alta	Auditoría	PU-22, PU-23 PI-02, PI-07 PS-16 PA-15
STORE-2 3	Filtrar registros de auditoría	<i>Administradora</i>	Media	Auditoría	PS-16 PA-15
STORE-2 4	Generar reporte mensual de auditoría	<i>Dueña</i>	Media	Auditoría	PS-16 PA-15

5.1 Criterios de Aceptación Clave

Los criterios de aceptación completos de cada historia se encuentran en Jira. A continuación se destacan los criterios que tienen mayor impacto en el diseño de los casos de prueba.

ID	Historia	Criterios de aceptación clave para las pruebas	Casos de prueba que los verifican
STORE-01	Registrar venta	- Se almacenan producto, cantidad, precio unitario, precio total y hora. - El tiempo de respuesta es menor a 2 segundos. - No es posible registrar la venta si el stock es insuficiente. - El usuario ve una confirmación visual del registro exitoso.	PU-08, PU-09, PU-10 PI-03, PI-04 PS-01, PS-02 (rendimiento) PA-02, PA-03
STORE-04	Anular venta	- Solo usuarios con rol administradora pueden anular ventas. - Un cajero que intente anular ve la acción	PU-13 PI-07 PS-05 (funcional), PS-06 (control acceso) PA-06

		bloqueada con mensaje de acceso restringido. • Cada anulación registra automáticamente usuario, fecha, motivo y venta afectada.	
STORE-06	Actualizar inventario tras venta	• Al confirmar una venta, el stock disminuye en tiempo real. • No es posible registrar una venta si el stock es insuficiente. • Tras ventas consecutivas el inventario refleja exactamente las unidades descontadas.	PU-16 PI-03 PS-01, PS-08 PA-02
STORE-07	Alerta de stock mínimo	• Cuando el stock llega al umbral mínimo configurado, se genera una alerta automática en la app. • La alerta indica nombre del producto, stock actual y fecha del aviso.	PU-17 PI-09 PS-09 PA-08
STORE-13	Restringir acceso según rol	• La restricción aplica tanto en la interfaz visual como en la API. • El intento de acceso no autorizado queda registrado en el log de auditoría. • El sistema no revela información parcial al usuario no autorizado.	PU-07 PI-12 PS-14, PS-15 PA-13, PA-14
STORE-15	Alerta caída en ventas	• Si la caída supera el 20%, se genera una alerta automática en el panel. • La alerta muestra porcentaje de variación, ventas del periodo actual y del anterior.	PU-20 PS-11 PA-10
STORE-22	Registrar acciones críticas	• Se registran automáticamente: anulaciones, modificaciones de inventario, creación de usuarios e intentos fallidos de inicio de sesión. • Cada registro incluye usuario, acción	PU-22, PU-23 PI-02, PI-07 PS-16 PA-15

		realizada, fecha y hora. • El registro es automático y no puede ser desactivado por ningún usuario.	
--	--	--	--

6. Objetivos del plan de pruebas

6.1 Contexto del Proyecto

StoreVision es un sistema de gestión diseñado para tiendas de abarrotes. Permite registrar ventas, controlar el inventario, generar reportes económicos y gestionar usuarios con diferentes niveles de acceso. El sistema está desarrollado siguiendo el patrón de diseño MVC (Modelo–Vista–Controlador), que organiza el código en tres responsabilidades bien diferenciadas.

6.2 Framework y Tecnología Principal

El sistema está construido con Python como lenguaje de programación y utiliza los siguientes componentes tecnológicos centrales:

Lenguaje	Python 3 — lenguaje de programación principal del proyecto.
Framework Web	FastAPI — framework de Python que recibe las solicitudes del navegador y las dirige a la lógica del sistema. Gestiona rutas, autenticación de sesiones y formatos de respuesta.
ORM / BD	SQLAlchemy — librería que permite trabajar con la base de datos usando código Python en lugar de SQL directo. La base de datos utilizada es SQLite, un archivo local para desarrollo.
Plantillas	Jinja2 — motor de plantillas que genera las páginas HTML dinámicas que ve el usuario final en el navegador.

Cifrado	Passlib con bcrypt — librería encargada de cifrar las contraseñas de los usuarios de forma segura antes de guardarlas en la base de datos.
Servidor	Uvicorn — servidor que pone en marcha la aplicación FastAPI y la hace accesible desde el navegador.

6.3 Estructura MVC del Proyecto

El patrón MVC separa el sistema en tres capas con responsabilidades distintas. A continuación se describe qué archivos conforman cada capa y qué función cumple cada uno:

6.3.1 *MODELO — La capa de datos*

El Modelo representa la estructura de la información que maneja el sistema. Define cómo se organizan los datos y cómo se almacenan en la base de datos. Es la capa más cercana a los datos reales del negocio.

Archivo	Responsabilidad
modelos.py	Define las tablas de la base de datos como clases de Python. Contiene: Usuario (datos de los empleados), Producto (catálogo e inventario), Sucursal, Venta (cabecera de cada transacción), ItemVenta (ítems de cada venta), MovimientoInventario (entradas y salidas de stock) y RegistroAuditoria (historial de acciones).
database.py	Configura la conexión con la base de datos SQLite, crea las tablas al iniciar la aplicación y proporciona la sesión de base de datos que usan los controladores para leer y escribir información.

6.3.2 CONTROLADOR — *La capa de lógica de negocio*

El Controlador contiene toda la lógica del sistema: las reglas de negocio, los cálculos, las validaciones y la coordinación de operaciones. Es el cerebro de la aplicación; no sabe cómo se ve la interfaz ni cómo está organizada la base de datos, solo sabe qué hacer con los datos.

Archivo	Responsabilidad y Funciones Principales
auth_controller.py	Gestiona el acceso al sistema. Sus funciones principales son: verificar_password (compara contraseñas), obtener_hash_password (cifra contraseñas), autenticar_usuario (valida credenciales y registra el intento en auditoría) y crear_usuario (registra nuevos empleados).
ventas_controller.py	Gestiona las transacciones de venta. Sus funciones principales son: registrar_venta (valida stock, calcula totales, crea la venta y descuenta inventario), anular_venta (revierte una venta y restaura stock), obtener_ventas_por_periodo (consulta ventas por rango de fechas) y consolidar_ventas_diarias (resumen del día).
inventario_controller.py	Gestiona el inventario de productos. Sus funciones principales son: registrar_movimiento (registra entradas y salidas de stock), verificar_alertas_inventario (detecta productos bajo el mínimo), obtener_historial_movimientos (consulta los movimientos de un producto) y obtener_productos_mas_vendidos (ranking de ventas).

reportes_controller.py	Genera los reportes y análisis del negocio. Sus funciones principales son: generar_balance_economico (calcula ventas, costos, utilidad y margen), obtener_indicadores_ventas (compara periodos y detecta caídas) y obtener_productos_mas_vendidos (ranking de productos por periodo).
------------------------	---

6.3.3 VISTA — La capa de interfaz de usuario

La Vista está contenida íntegramente en un único archivo: index.html. Este archivo centraliza en un solo lugar toda la interfaz de usuario, toda la lógica de navegación y toda la comunicación con el servidor. No existen archivos CSS ni JavaScript separados; los estilos se aplican de forma inline y el código JavaScript está embebido directamente al final del archivo dentro de un bloque de script.

La navegación entre módulos no recarga la página. En su lugar, todos los módulos están presentes en el HTML al mismo tiempo y el sistema muestra u oculta cada sección según el botón de navegación que el usuario presione (sistema de pestañas o tabs). Esto hace que la aplicación se comporte como una página única (Single Page Application).

Sección dentro de index.html	Contenido y responsabilidad
Barra de navegación	Muestra el nombre del sistema, los botones para cambiar entre módulos (Dashboard, Ventas, Inventario, Reportes), el nombre del usuario activo y el botón de cierre de sesión. Es visible en todo

	momento independientemente del módulo activo.
Sección de Login	Formulario de inicio de sesión con campos de correo y contraseña. Se muestra cuando no hay sesión activa y se oculta automáticamente tras autenticarse correctamente.
Tab — Dashboard	Primera pestaña visible tras iniciar sesión. Muestra tres indicadores del día: número de ventas realizadas, monto total recaudado y cantidad de alertas de inventario activas. Incluye un botón para actualizar los datos.
Tab — Ventas	Contiene el formulario para registrar nuevas ventas (selector de productos y cantidades), la tabla de ventas del día con filtro por fecha, y el panel de anulación de ventas con campo de motivo. El botón de anulación solo aparece para la administradora.
Tab — Inventario	Contiene cuatro secciones: alertas de stock bajo, tabla de productos con su stock actual y mínimo, formulario para registrar movimientos de entrada o salida, formulario para crear nuevos productos, e historial de movimientos filtrable por rango de fechas.
Tab — Reportes	Contiene filtros de fechas y tres secciones de análisis: balance económico (ventas, utilidad y margen), indicadores comparativos de ventas entre periodos con alerta de caída, y tabla del ranking de productos más vendidos por unidades e ingresos.
Bloque JavaScript (embebido)	Código JavaScript incluido al final del propio index.html. Gestiona: la verificación de sesión al cargar la página, el inicio y cierre de sesión, el control de visibilidad de pestañas, la comunicación con la API del servidor para cargar y enviar datos, el control de acceso por rol en la interfaz (mostrar u ocultar el botón de anulación según el

	rol del usuario) y la actualización dinámica de todas las tablas y métricas.
--	--

6.3.4 RUTAS API — *El puente entre Vista y Controlador*

Existe un archivo adicional que actúa como puente entre las peticiones del navegador y la lógica de negocio:

Archivo	Responsabilidad
api_views.py	Define todas las rutas de la aplicación (las direcciones que usa el navegador para comunicarse con el servidor). Recibe cada solicitud, verifica que el usuario tenga sesión y permisos, llama al controlador correspondiente y devuelve la respuesta. Es la capa de enrutamiento y seguridad del sistema.

6.4 Roles de Usuario

El sistema contempla dos roles con permisos diferenciados, relevantes para las pruebas:

Rol	Permisos y Acceso
Administradora	Acceso completo al sistema: registrar ventas, anular ventas, gestionar inventario, crear productos, generar reportes y ver auditoría.
Cajero	Acceso limitado: puede registrar ventas y consultar inventario. No puede anular ventas, crear productos ni acceder al módulo de reportes.

6.5 Niveles de Prueba y su Relación con el MVC

Las pruebas del sistema se organizan en cuatro niveles de menor a mayor alcance. Cada nivel verifica una capa o combinación de capas del patrón MVC:

Nivel	Qué verifica	Capa(s) del MVC	¿Qué lo hace diferente?
Unitarias	Funciones individuales de los controladores y modelos de forma aislada.	Controladores · Modelos	Las funciones se prueban solas, simulando la base de datos para no depender de ella.
Integración	La comunicación entre controladores, modelos y base de datos real.	Controladores ↔ Modelos ↔ Base de Datos	Se usa una base de datos de prueba real para verificar que las transacciones son correctas y atómicas.
Sistema	Los flujos completos de la aplicación desde la solicitud hasta la respuesta final.	Rutas API → Controladores → Modelos → BD	Se prueba el sistema como un todo, verificando que cada operación funciona de extremo a extremo.
Aceptación	Que el sistema cumple los criterios de negocio desde la perspectiva del	Sistema completo + Interfaz de usuario	Se valida que el usuario puede realizar sus tareas del día a día de forma correcta y sin

	usuario.		dificultades.
--	----------	--	---------------

6.6 Objetivos de Pruebas Unitarias

Las pruebas unitarias verifican el comportamiento de funciones individuales de los controladores y del modelo de datos de forma completamente aislada. Para ello, se simula la base de datos con objetos ficticios, de modo que la prueba depende únicamente de la lógica interna de la función y no de factores externos. Son las pruebas más rápidas de ejecutar y las primeras en detectar errores de lógica.

Alcance: Funciones de `auth_controller.py` · `ventas_controller.py` · `inventario_controller.py` · `reportes_controller.py` · `modelos.py`

ID	Área / Componente	Objetivo de Prueba	RFs Asociados	Criterio de Éxito
PU-01	Controlador de Autenticación	Verificar que la función de verificación de contraseña (<code>verificar_password</code>) reconoce correctamente si una contraseña ingresada coincide o no con la contraseña cifrada almacenada, sin necesidad de consultar la base de datos.	RF1.4, RF4.1	Retorna verdadero para contraseña correcta y falso para incorrecta en todos los casos evaluados.
PU-02	Controlador de Autenticación	Verificar que la función de cifrado de contraseñas (<code>obtener_hash_password</code>) genera un código cifrado seguro que es único e irrepetible, incluso cuando se cifra la misma contraseña dos veces.	RF4.1	El código cifrado nunca es igual a la contraseña original; dos cifrados de la misma clave producen resultados diferentes.
PU-03	Controlador de Autenticación	Verificar que cuando se intenta iniciar sesión con credenciales	RF1.4,	No se concede acceso y queda registrado

		incorrectas, la función <code>autenticar_usuario</code> rechaza el acceso y registra automáticamente un intento fallido en el historial de auditoría.	RF8.1	exactamente un evento de 'login fallido' en auditoría.
PU-04	Controlador de Autenticación	Verificar que cuando las credenciales son correctas y el usuario está activo, la función <code>autenticar_usuario</code> concede el acceso y registra el ingreso exitoso en el historial de auditoría.	RF4.1, RF8.1	Se retorna el usuario autenticado y queda registrado un evento de 'login exitoso' en auditoría.
PU-05	Controlador de Autenticación	Verificar que la función de creación de usuario (<code>crear_usuario</code>) impide registrar un correo electrónico que ya existe en el sistema, retornando un mensaje de error claro.	RF4.1	Se retorna un mensaje de error indicando que el correo ya está registrado; no se crea ningún usuario duplicado.
PU-06	Controlador de Ventas	Verificar que la función de registro de venta (<code>registrar_venta</code>) rechaza la operación y retorna un mensaje de error cuando se intenta crear una venta sin agregar ningún producto.	RF1.1	Se retorna un error descriptivo; no se modifica ningún dato en la base de datos.
PU-07	Controlador de Ventas	Verificar que la función <code>registrar_venta</code> detecta cuando la cantidad solicitada de un producto supera el inventario disponible y retorna un error informando la situación, sin modificar el stock.	RF1.1, RF2.2	Se retorna un error de stock insuficiente; el stock del producto permanece igual.
PU-08	Controlador de	Verificar que la función	RF1.1	El total calculado

	Ventas	registrar_venta calcula correctamente el total de la venta sumando los subtotales de cada producto (cantidad multiplicada por precio unitario).		coincide con la suma aritmética de los subtotales con una tolerancia máxima de \$0.01.
PU-09	Controlador de Ventas	Verificar que la función de anulación de venta (anular_venta) detecta cuando una venta ya fue previamente anulada y retorna un error indicándolo, sin realizar cambios adicionales.	RF1.4	Se retorna un mensaje de error señalando que la venta ya está anulada; el estado no cambia.
PU-10	Controlador de Inventario	Verificar que la función de registro de movimiento (registrar_movimiento) actualiza correctamente el stock de un producto cuando se registra una entrada de mercancía, sumando la cantidad indicada al stock existente.	RF2.1, RF2.2	El stock resultante es exactamente igual al stock anterior más la cantidad ingresada.
PU-11	Controlador de Inventario	Verificar que la función registrar_movimiento impide registrar una salida de mercancía cuando la cantidad a retirar supera el stock disponible del producto, retornando un error claro.	RF2.1, RF2.2	Se retorna un error de stock insuficiente; el stock del producto permanece sin cambios.
PU-12	Controlador de Inventario	Verificar que la función de revisión de alertas (verificar_alertas_inventario) identifica y retorna únicamente los productos cuyo stock actual es igual o inferior al mínimo configurado,	RF2.3, RF5.1	La lista retornada contiene solo los productos por debajo del umbral; ningún producto con stock suficiente aparece en

		sin incluir productos con stock normal.		la lista.
PU-13	Controlador de Reportes	Verificar que la función de balance económico (generar_balance_economico) calcula correctamente la utilidad bruta restando el costo de ventas al total de ingresos.	RF3.3	La utilidad bruta calculada coincide con la diferencia entre ventas totales y costos, con precisión de dos decimales.
PU-14	Controlador de Reportes	Verificar que la función de indicadores de ventas (obtener_indicadores_ventas) calcula correctamente el porcentaje de variación entre el periodo actual y el anterior, y activa la alerta de caída cuando la disminución supera el 15%.	RF3.1, RF5.2	El porcentaje de variación es aritméticamente correcto; la alerta se activa cuando la caída supera el 15% y no se activa cuando no aplica.
PU-15	Modelo — Producto	Verificar que el modelo de datos del producto almacena y recupera correctamente todos sus campos obligatorios (código, nombre, precio de venta, costo, stock actual y stock mínimo) sin pérdida ni alteración de información.	RF2.1	Todos los campos persisten con los valores originales sin truncamiento ni modificación.
PU-16	Modelo — Registro de Auditoría	Verificar que el modelo de auditoría (RegistroAuditoria) permite guardar registros sin un usuario asociado, lo que es necesario para documentar intentos de acceso fallidos de personas no identificadas.	RF8.1	El registro se guarda exitosamente con el campo de usuario en blanco, sin violar restricciones del modelo de datos.

Cobertura mínima esperada	80% de las líneas de código en cada controlador. Las funciones de cálculo financiero (balance, total de venta, variación porcentual) deben alcanzar cobertura del 100%.
---------------------------	---

6.7 Objetivos de Pruebas de Integración

Las pruebas de integración verifican que las distintas capas del sistema funcionan correctamente en conjunto. A diferencia de las pruebas unitarias, aquí se usa una base de datos de prueba real (no simulada) para comprobar que los controladores interactúan correctamente con el modelo de datos, que las transacciones son atómicas (todo o nada) y que los efectos secundarios (como actualizar el stock al registrar una venta) se producen correctamente.

Alcance: Controladores ↔ Modelos ↔ Base de datos de prueba (SQLite en memoria)

ID	Área / Componente	Objetivo de Prueba	RFs Asociados	Criterio de Éxito
PI-01	Autenticación ↔ Base de Datos	Verificar que el flujo completo de inicio de sesión consulta la base de datos real, valida las credenciales y guarda el registro de auditoría en la misma operación, sin dejar datos incompletos.	RF1.4, RF8.1	El usuario es retornado correctamente y el registro de auditoría queda visible en la base de datos de prueba.
PI-02	Autenticación ↔ Base de Datos	Verificar que al crear un nuevo usuario, el proceso guarda el registro con contraseña cifrada en la tabla de usuarios y registra la acción de creación en auditoría, todo en una única operación que se revierte por	RF4.1, RF8.1	Registro en la tabla de usuarios y registro de auditoría creados; si ocurre un error, ninguno de los dos queda guardado.

		completo si ocurre algún error.		
PI-03	Ventas ↔ Inventario ↔ Base de Datos	Verificar que al registrar una venta, el sistema descuenta el stock del producto y crea el movimiento de inventario de salida simultáneamente, de forma que si algo falla, ni la venta ni el movimiento queden guardados.	<i>RF1.1,</i> <i>RF2.2</i>	Stock decrementado y movimiento de salida creado; ante cualquier error, ambas operaciones se revierten completamente.
PI-04	Ventas ↔ Inventario ↔ Base de Datos	Verificar que al anular una venta, el sistema devuelve el stock de todos los productos incluidos en ella y crea un movimiento de entrada por cada uno, reflejando la reposición correctamente en el inventario.	<i>RF1.4,</i> <i>RF2.1</i>	Stock restaurado para cada producto; existe un movimiento de entrada por cada ítem anulado en el historial de inventario.
PI-05	Inventario ↔ Base de Datos	Verificar que al registrar un movimiento de inventario, el producto queda actualizado con el nuevo stock y el registro del movimiento muestra correctamente el stock antes y después de la operación.	<i>RF2.1,</i> <i>RF2.4</i>	El producto refleja el stock actualizado; el movimiento registrado contiene los valores de stock anterior y stock nuevo correctos.
PI-06	Inventario ↔ Alertas	Verificar que después de registrar una salida de mercancía que lleva el stock de un producto por debajo del mínimo, la función de revisión de alertas consulta la base de datos y retorna ese producto como alerta.	<i>RF2.3,</i> <i>RF5.1</i>	La lista de alertas refleja el estado real del inventario en la base de datos tras el movimiento de salida.
PI-07	Reportes ↔ Ventas ↔ Base	Verificar que la generación del balance económico obtiene los datos	<i>RF3.3</i>	Los valores del balance coinciden con

	de Datos	reales de ventas, ítems vendidos y costos de productos desde la base de datos, y calcula valores coherentes con los registros existentes.		la suma manual de los registros de ventas en la base de datos de prueba.
PI-08	Reportes ↔ Base de Datos	Verificar que la función de productos más vendidos (obtener_productos_mas_vendidos) cruza correctamente las tablas de ventas, ítems de venta y productos, retornando el ranking ordenado por cantidad vendida de mayor a menor.	<i>RF3.2</i>	El orden del ranking es correcto de mayor a menor; los productos de ventas anuladas no aparecen en el resultado.
PI-09	Controladores ↔ Base de Datos — Atomicidad	Verificar que si ocurre un error durante el registro de una venta (por ejemplo, un producto no encontrado al procesar el segundo ítem), la operación completa se revierte y no quedan registros parciales de la venta, sus ítems, movimientos de inventario ni auditoría.	<i>RF1.1</i>	Ninguna tabla contiene filas incompletas o huérfanas tras el error; la base de datos queda exactamente como estaba antes.
PI-10	Rutas API ↔ Controladores	Verificar que las funciones de enrutamiento de api_views.py crean correctamente los controladores, les pasan los parámetros necesarios y devuelven la respuesta del controlador sin transformaciones incorrectas al usuario.	<i>RF1.1– RF3.3</i>	La respuesta enviada al cliente corresponde exactamente al resultado producido por el controlador para cada operación principal.
PI-11	Sesiones ↔ Controladores	Verificar que las operaciones protegidas rechazan solicitudes sin sesión iniciada, y que las funciones reservadas para la administradora	<i>RF4.2, RF1.4</i>	Solicitudes sin sesión reciben error de no autenticado; solicitudes del cajero

		rechazan al cajero, devolviendo mensajes de error apropiados en ambos casos.		en funciones restringidas reciben error de permisos insuficientes.
--	--	--	--	--

Entorno de prueba	Se utiliza una base de datos SQLite en memoria o una base de datos de prueba aislada. Las tablas se recrean antes de cada conjunto de pruebas para garantizar un estado limpio.
--------------------------	---

6.8 Objetivos de Pruebas de Sistema

Las pruebas de sistema validan los flujos completos de negocio de extremo a extremo, ejerciendo el sistema tal como lo haría un usuario real a través del navegador. Se verifica que la cadena completa funciona correctamente: desde que llega una solicitud al servidor, pasa por las rutas de `api_views.py`, es procesada por el controlador correspondiente, interactúa con la base de datos y retorna la respuesta correcta al cliente.

Alcance: Capa de rutas (`api_views.py`) → Controladores → Modelos → Base de datos + Pantallas HTML

ID	Área / Componente	Objetivo de Prueba	RFs Asociados	Criterio de Éxito
PS-01	Registro de Ventas (RF1)	Verificar el flujo completo de registro de una venta: el usuario inicia sesión, selecciona los productos, confirma la operación, el sistema la registra exitosamente, descuenta el stock de cada producto y guarda el evento en el historial de auditoría.	RF1.1, RF2.2, RF8.1	La venta queda registrada en la base de datos; el stock está decrementado; existe el registro de auditoría correspondiente.

PS-0 2	Consulta de Ventas (RF1)	Verificar que la función de consulta de ventas por fecha retorna correctamente las ventas del día indicado y que la pantalla de ventas las muestra en la tabla de resultados.	RF1.3	Solo aparecen las ventas correspondientes al rango de fecha solicitado, sin incluir ventas de otros días.
PS-0 3	Anulación de Ventas (RF1)	Verificar que la función de anulación de ventas revierte el stock, cambia el estado de la venta a 'anulada' y está disponible únicamente para la administradora; el cajero no puede ejecutarla.	RF1.4, RF4.2	La venta queda anulada y el stock restaurado para la administradora; el cajero recibe un error de permisos insuficientes.
PS-0 4	Movimientos de Inventario (RF2)	Verificar el flujo completo de registro de una entrada de mercancía: la administradora registra el movimiento, el stock del producto aumenta correctamente y el movimiento aparece en el historial de inventario.	RF2.1, RF2.4	El stock incrementa con el valor correcto; el movimiento aparece en la consulta del historial de inventario.
PS-0 5	Alertas de Inventario (RF2)	Verificar que la función de alertas retorna los productos con stock bajo el mínimo y que tanto el panel principal (dashboard) como la pantalla de inventario los muestran como alertas visibles.	RF2.3, RF5.1	Los productos bajo el umbral aparecen en la lista de alertas; el contador del dashboard refleja la cantidad correcta de alertas.
PS-0 6	Creación de Productos (RF2)	Verificar que la administradora puede crear un producto nuevo a través del formulario y que el producto queda	RF2.1, RF4.2	El producto aparece en el inventario y en el formulario de ventas;

		disponible en el listado de inventario y en el selector de productos de la pantalla de ventas. El cajero no puede acceder a esta función.		el cajero recibe un error de permisos al intentarlo.
PS-0 7	Balance Económico (RF3)	Verificar que la función de balance económico (generar_balance_economico) retorna la estructura completa con ventas totales, costo de ventas, utilidad bruta y margen de utilidad, y que la pantalla de reportes muestra estas métricas correctamente.	RF3.3	La estructura de respuesta contiene todos los campos esperados con valores calculados consistentes; la interfaz muestra las métricas.
PS-0 8	Productos Más Vendidos (RF3)	Verificar que la función de productos más vendidos (obtener_productos_mas_vendidos) retorna la lista ordenada por unidades vendidas y que la pantalla de reportes muestra el ranking correctamente.	RF3.2	La lista está en orden descendente correcto; la pantalla de reportes presenta el ranking de manera comprensible.
PS-0 9	Indicadores de Ventas (RF3)	Verificar que la función de indicadores de ventas (obtener_indicadores_ventas) retorna la variación porcentual entre periodos y activa la señal de alerta de caída cuando la disminución supera el 15%.	RF3.1, RF5.2	El porcentaje de variación es correcto; la alerta de caída refleja fielmente la regla de negocio establecida.
PS-1 0	Control de Roles en Pantalla (RF4)	Verificar que la pantalla de reportes muestra su contenido completo solo cuando el usuario es administradora, y muestra un mensaje de acceso restringido cuando el usuario es	RF4.2	El contenido de reportes es visible para la administradora; el cajero ve únicamente el mensaje de

		cajero.		restricción de acceso.
PS-1 1	Alerta de Inventario Bajo — Flujo Completo (RF5)	Verificar el flujo de alerta de inventario de extremo a extremo: se registra una venta que lleva el stock por debajo del mínimo, la función de alertas detecta el producto y el dashboard muestra el contador de alertas actualizado.	RF2.3, RF5.1	El producto aparece en la lista de alertas; el contador del dashboard es mayor a cero; la alerta se muestra en la pantalla de inventario.
PS-1 2	Consolidado Diario (RF1.2)	Verificar que la función de consolidado diario (consolidar_ventas_diarias) retorna el número y monto total de ventas del día de forma coherente con las ventas registradas, y que el dashboard los muestra correctamente.	RF1.2	Los valores del consolidado coinciden con la suma real de ventas completadas en el día.
PS-1 3	Registro de Auditoría — Sistema Completo (RF8)	Verificar que todas las acciones críticas del sistema (inicio de sesión, registro de venta, anulación, movimiento de inventario y creación de producto) generan automáticamente un registro en el historial de auditoría con el usuario responsable, el tipo de acción, la descripción y la fecha.	RF8.1, RF8.2	Existe exactamente un registro de auditoría por cada acción ejecutada; todos los campos están completos y son correctos.
PS-1 4	Rendimiento en Registro de Ventas (RNF1)	Verificar que el sistema responde en un tiempo razonable al registrar una venta bajo condiciones normales de operación, cumpliendo con el límite de tiempo establecido en los	RF1.1, RNF1.2	El tiempo de respuesta es inferior a 2 segundos en al menos el 95% de las operaciones de registro de venta

		requerimientos no funcionales.		ejecutadas consecutivamente.
--	--	--------------------------------	--	------------------------------

Datos de prueba	La base de datos se puebla con los productos y usuarios iniciales del script de inicialización antes de ejecutar cada conjunto de pruebas de sistema.
Manejo de sesiones	Se simula el flujo de inicio de sesión para obtener un identificador de sesión válido antes de invocar operaciones protegidas. La sesión se reutiliza dentro del mismo conjunto de pruebas.

6.9 Objetivos de Pruebas de Aceptación

Las pruebas de aceptación validan que el sistema cumple los criterios de aceptación definidos en los requerimientos funcionales desde la perspectiva del usuario final — la administradora y el cajero. A diferencia de los niveles anteriores, el foco no está en la implementación técnica sino en comprobar que el sistema hace lo que el negocio necesita, de la forma en que el usuario lo esperaría.

Alcance: Sistema completo en entorno de prueba — Perspectiva de la administradora y el cajero

ID	Área / Componente	Objetivo de Prueba	RFs Asociados	Criterio de Éxito
PA-01	Registro de Ventas (RF1)	El cajero puede iniciar sesión, seleccionar los productos disponibles, confirmar la venta y recibir la confirmación en pantalla; el sistema descuenta el stock automáticamente sin que el cajero deba hacer nada adicional.	RF1.1, RF2.2	La venta se registra exitosamente en tres pasos o menos desde el inicio de sesión; el stock se actualiza y aparece un mensaje de confirmación.
PA-02	Consulta por	La administradora puede filtrar el	RF1.3	Solo aparecen en

	Periodo (RF1)	historial de ventas seleccionando un rango de fechas y visualizar únicamente las ventas correspondientes a ese periodo en la tabla de resultados.		pantalla las ventas del rango de fechas seleccionado, sin mezclar ventas de otras fechas.
PA-03	Anulación de Venta (RF1)	La administradora puede anular una venta existente indicando el motivo de la anulación; el sistema devuelve el stock al inventario y registra la acción. El cajero no tiene acceso a esta función.	<i>RF1.4,</i> <i>RF4.2</i>	La anulación se completa exitosamente para la administradora; el cajero no puede ver ni ejecutar esta opción.
PA-04	Gestión de Inventario (RF2)	La administradora puede registrar la entrada de nueva mercancía para un producto y verificar en la tabla de inventario que el stock aumentó, y en el historial que aparece el movimiento con el motivo y el nombre del usuario que lo registró.	<i>RF2.1,</i> <i>RF2.4</i>	El stock actualizado es visible en la lista de inventario; el historial muestra la fila del movimiento con todos sus datos.
PA-05	Alertas de Inventario (RF2)	Después de registrar ventas que llevan un producto por debajo del stock mínimo, la sección de alertas en la pantalla de inventario y el contador del dashboard se actualizan automáticamente mostrando dicho producto como alerta.	<i>RF2.3,</i> <i>RF5.1</i>	La alerta aparece visible sin que el usuario deba hacer ninguna acción adicional tras la venta.
PA-06	Creación de Producto (RF2)	La administradora puede agregar un nuevo producto usando el formulario de inventario y encontrarlo disponible	<i>RF2.1,</i> <i>RF4.2</i>	El producto aparece en ambas pantallas; el cajero no puede acceder al formulario

		inmediatamente en el listado de inventario y en el formulario de ventas para ser seleccionado.		de creación.
PA-07	Balance Económico (RF3)	La administradora puede generar el balance económico del periodo seleccionado y visualizar en la pantalla de reportes las métricas de ventas totales, costo, utilidad bruta y margen de utilidad de forma clara.	<i>RF3.3,</i> <i>RF4.2</i>	Las métricas son visibles, comprensibles y coherentes; el cajero no puede acceder a la pantalla de reportes.
PA-08	Productos Más Vendidos (RF3)	La pantalla de reportes muestra el ranking de productos más vendidos ordenado por unidades, con código, categoría y total de ingresos, para el rango de fechas seleccionado.	<i>RF3.2</i>	El ranking está en orden correcto de mayor a menor; no aparecen productos con cero ventas en el listado.
PA-09	Indicadores Comparativos (RF3)	El sistema muestra la variación porcentual de ventas entre el periodo seleccionado y el anterior, y presenta una señal de alerta visible cuando la caída supera el 15%.	<i>RF3.1,</i> <i>RF5.2</i>	El porcentaje se muestra correctamente; la alerta aparece destacada en rojo cuando aplica la regla de negocio.
PA-10	Control de Roles (RF4)	Un usuario con rol de cajero no puede ver los reportes, no puede crear productos ni anular ventas. Una administradora puede acceder a todas las funciones del sistema sin restricciones ni mensajes de error.	<i>RF4.1,</i> <i>RF4.2</i>	El cajero es bloqueado en cada módulo restringido; la administradora completa todos los flujos sin inconvenientes.

PA-11	Dashboard — Resumen del Día (RF5)	El dashboard principal muestra al iniciar sesión el número de ventas del día, el monto total recaudado y el contador de alertas de inventario, reflejando de forma correcta el estado actual del negocio.	<i>RF1.2,</i> <i>RF5.1</i>	Los tres indicadores del dashboard son correctos y coherentes con los datos registrados en la sesión actual.
PA-12	Trazabilidad y Auditoría (RF8)	Cada acción realizada por cualquier usuario (inicio de sesión, registro de venta, anulación, movimiento de inventario) puede verificarse en el historial de auditoría con la fecha, el nombre del usuario y la descripción de lo ocurrido.	<i>RF8.1,</i> <i>RF8.3</i>	Todos los registros de auditoría están presentes y contienen información completa para cada flujo ejecutado.
PA-13	Usabilidad General (RNF3)	Un usuario sin experiencia previa en StoreVision puede completar los flujos de registro de venta, consulta de inventario y generación de reporte en tres pasos o menos, guiándose únicamente por la interfaz del sistema.	<i>RNF3.1,</i> <i>RNF3.3</i>	Los tres flujos se completan en máximo tres interacciones sin ayuda externa y sin errores visibles en la interfaz.
PA-14	Consolidado Diario (RF1.2)	Al cierre de la jornada, la administradora puede ver en el dashboard la cantidad y el monto total de ventas del día, y estos datos coinciden exactamente con las transacciones que fueron registradas durante la sesión.	<i>RF1.2</i>	Los datos del consolidado coinciden al 100% con las ventas registradas manualmente durante la jornada de prueba.

	seguridad se ejecutan con ambos roles para verificar que los controles funcionan en la práctica.
Criterio general de aprobación	El 100% de los objetivos de prioridad Alta deben aprobarse. Los objetivos de prioridad Media deben aprobarse en al menos el 90% de los casos evaluados.

6.10 Resumen de Objetivos y Trazabilidad

La siguiente tabla consolida la cantidad de objetivos definidos por nivel de prueba y su cobertura sobre los bloques de requerimientos funcionales documentados.

Nivel	Objetivos Definidos	RFs Cubiertos	Enfoque Principal
Unitarias	16	RF1, RF2, RF3, RF4, RF8	Lógica aislada de funciones individuales en controladores y modelos.
Integración	11	RF1, RF2, RF3, RF4, RF8	Comunicación entre controladores, modelos y base de datos real; atomicidad de transacciones.
Sistema	14	RF1–RF5, RF8, RNF1	Flujos completos de extremo a extremo; seguridad por rol; rendimiento básico.
Aceptación	14	RF1–RF5, RF8, RNF3	Criterios de negocio verificados desde la perspectiva del usuario final; usabilidad.
TOTAL	55	RF1–RF8, RNF1, RNF3	Cobertura completa de los requerimientos funcionales prioritarios definidos para StoreVision.

7. Clases y tipos de pruebas a realizar.

7.1 Clases y Tipos de Prueba

Esta sección define las clases y tipos de prueba que se aplican en el plan, y describe de manera general cómo se utilizan en cada nivel de prueba de StoreVision.

7.1.1 Clases de Prueba

La clase de prueba determina qué tanto conocimiento del código interno tiene el evaluador al momento de diseñar los casos.

Clase	Definición	Aplicación en StoreVision
Caja Blanca	Se conoce el código fuente y se diseña los casos para recorrer todas las ramas, condiciones y caminos internos del componente.	Se aplica en las pruebas unitarias, donde se accede directamente a las funciones de los controladores (auth_controller, ventas_controller, inventario_controller, reportes_controller) para verificar cada rama condicional y cada cálculo interno: por ejemplo, el cálculo de utilidad bruta, la activación de la alerta de caída de ventas o el manejo de la división por cero en el margen.
Caja Negra	El evaluador no conoce el código interno; solo conoce las entradas que acepta el componente y los resultados que debe producir.	Se aplica en las pruebas de sistema y de aceptación, donde se ejercen los flujos completos a través de la API o de la interfaz index.html sin importar cómo están implementados internamente: registrar una venta, anular con motivo, generar el balance económico o verificar que el cajero no puede acceder a reportes.
Caja Gris	Conocimiento parcial de la arquitectura: se sabe qué capas interactúan pero no los detalles de	Se aplica en las pruebas de integración, donde se conoce que el controlador llama al modelo y este escribe en la base de datos, pero no se evalúa el

	implementación de cada una.	SQL que SQLAlchemy genera. Se verifica la atomicidad de las transacciones y la consistencia entre tablas relacionadas (Venta, ItemVenta, Producto, MovimientoInventario, RegistroAuditoria).
--	-----------------------------	--

7.1.2 Tipos de Prueba

El tipo de prueba define qué característica del sistema se está verificando en cada caso.

Tipo	Qué verifica	Aplicación en StoreVision
Funcional	Que el sistema realiza correctamente las funciones de negocio definidas en los requerimientos.	Es el tipo predominante. Cubre los flujos de RF1 a RF8: registro y anulación de ventas, movimientos de inventario, alertas de stock bajo, balance económico, indicadores comparativos, ranking de productos y registro de auditoría.
Seguridad	Que los controles de acceso funcionan: usuarios sin autenticación o con rol insuficiente no pueden ejecutar operaciones restringidas.	Verifica que solicitudes sin session-id reciben HTTP 401, que el cajero recibe HTTP 403 al intentar anular ventas o crear productos, y que en la interfaz el botón de anulación y el formulario de nuevo producto no son visibles para el cajero.
Rendimiento	Que el sistema responde dentro de los tiempos máximos establecidos en los requerimientos no funcionales.	Se verifica el tiempo de respuesta al registrar una venta bajo carga normal: el percentil 95 debe ser inferior a 2 segundos, según el requerimiento no funcional de tiempo de respuesta.
Usabilidad	Que un usuario real puede	Verifica que los flujos principales de index.html

	completar las tareas del día a día de forma intuitiva, sin capacitación especializada.	(registrar venta, consultar inventario y generar balance) se completan en 3 interacciones o menos por un usuario sin experiencia previa, guiándose únicamente por la interfaz.
--	--	--

7.2 Aplicación por Nivel de Prueba

La siguiente tabla resume qué clase y tipos se aplican en cada nivel del plan, y la razón de esa combinación.

Nivel	Clase	Tipos	Justificación
Unitarias	Caja Blanca	Funcional	Se tiene acceso al código de cada función de los controladores. Los casos cubren todas las ramas condicionales y los cálculos internos de forma aislada, usando objetos simulados en lugar de la base de datos real.
Integración	Caja Gris	Funcional, Seguridad	Se conoce la arquitectura de capas pero se evalúa el comportamiento conjunto desde fuera de cada función individual. Se verifica que las transacciones sean atómicas y que las tablas relacionadas queden consistentes, usando una base de datos de prueba en memoria.
Sistema	Caja Negra	Funcional, Seguridad, Rendimiento	Se ejercen flujos completos a través de la API (api_views.py → controladores → modelos → BD) sin conocer la implementación interna. Se incluyen pruebas de control de roles a nivel HTTP y la verificación del tiempo de respuesta.
Aceptación	Caja Negra	Funcional,	El usuario final valida que el sistema cumple los

		Seguridad, Usabilidad	criterios del negocio operando directamente sobre index.html. Se verifica el control de acceso visible en la interfaz (botones u ocultos según el rol) y que los flujos principales son intuitivos.
--	--	--------------------------	---

8. Herramientas a utilizar para las pruebas

8.1 Herramientas de Prueba

Esta sección describe las herramientas seleccionadas para ejecutar las pruebas del sistema StoreVision y explica cómo se utiliza cada una. La selección se basa en las tecnologías con las que está construido el proyecto y en las prácticas recomendadas para este tipo de sistemas.

8.2 Resumen de Herramientas por Nivel

Herramienta	Nivel de prueba	Tipo de prueba	Propósito principal
pytest	Unitarias	Funcional	Ejecución de pruebas automáticas sobre cada función individual del sistema, verificando que produce el resultado correcto en distintas situaciones.
pytest + SQLite en memoria	Integración	Funcional, Seguridad	Verificación de que las distintas partes del sistema trabajan correctamente juntas, usando una base de datos temporal que no afecta los

			datos reales.
pytest + HTTPX	Sistema	Funcional, Seguridad, Rendimiento	Pruebas del sistema completo enviando solicitudes automáticas como si fueran peticiones reales de usuarios, verificando respuestas y tiempos.

8.3 Descripción de cada Herramienta

pytest <i>v8.x</i>
Nivel: Unitarias Clase de prueba: Caja Blanca
Descripción: Herramienta que permite ejecutar pruebas automáticas sobre el código Python del sistema. Cada prueba verifica que una función específica produce el resultado esperado ante distintas situaciones. Es la herramienta más utilizada en proyectos Python para este tipo de verificaciones.
Aplicación en StoreVision: • Cada función del sistema (iniciar sesión, registrar venta, calcular balance, etc.) se prueba de forma individual para confirmar que hace exactamente lo que debe hacer, sin depender de la base de datos real.
• Se simula la base de datos con datos de prueba controlados, de modo que cada caso comienza desde cero y los resultados no se mezclan entre sí.
• Se verifican tanto los casos normales como los casos de error: qué ocurre cuando el stock es insuficiente, cuando el correo ya está registrado o cuando se intenta anular una venta que no existe.
• Se mide qué porcentaje del código queda cubierto por las pruebas, para identificar partes del sistema que aún no han sido verificadas.

	Nivel: Integración Clase de prueba: Caja Gris
	Descripción: Combina la herramienta de pruebas pytest con una base de datos temporal que solo existe mientras se ejecutan las pruebas y desaparece al terminar. Esto permite probar cómo se comunican las distintas partes del sistema entre sí, sin afectar la base de datos real del negocio.
	Aplicación en StoreVision: • Al iniciar cada prueba se crea una base de datos temporal con todas las tablas del sistema, y al terminar se elimina automáticamente, dejando siempre un punto de partida limpio.
	• Se verifica que al registrar una venta, si algo falla en el proceso (por ejemplo, un producto no tiene stock suficiente), el sistema cancela toda la operación completa: no queda registrada la venta ni se descuenta ningún producto del inventario.
	• Se comprueba que al anular una venta, el stock de cada producto vuelve exactamente a la cantidad que tenía antes, y que el movimiento de devolución queda registrado correctamente.
	• Se valida que cada acción importante del sistema (iniciar sesión, registrar una venta, crear un usuario) deja una anotación en el registro de auditoría con el usuario responsable y la fecha.

	pytest + HTTPX <i>pytest v8.x / HTTPX v0.27.x</i>
	Nivel: Sistema Clase de prueba: Caja Negra
	Descripción: Herramienta que permite enviar solicitudes al sistema como si fueran peticiones reales desde un navegador o una aplicación, pero de forma automática y sin necesidad de abrir el sistema manualmente. Así se puede probar toda la cadena completa: desde que llega una solicitud hasta que el dato queda guardado.
	Aplicación en StoreVision: • Se envían solicitudes automáticas a todos los servicios del sistema (registrar venta, consultar inventario, generar balance, etc.) y se verifica que cada uno responde correctamente con los datos esperados.
	• Se comprueba el control de acceso: si alguien intenta usar el sistema sin haber iniciado sesión, el sistema le niega el acceso; si un cajero intenta anular una venta o crear un producto, el

sistema también se lo impide.

- Se prueban flujos completos de forma automática: iniciar sesión, registrar una venta con varios productos, verificar que el inventario bajó, anular la venta y confirmar que el inventario volvió a su estado original.

- Se mide el tiempo que tarda el sistema en registrar una venta para verificar que responde en menos de 2 segundos en condiciones normales de uso.

AVANCES SEGUNDO CORTE

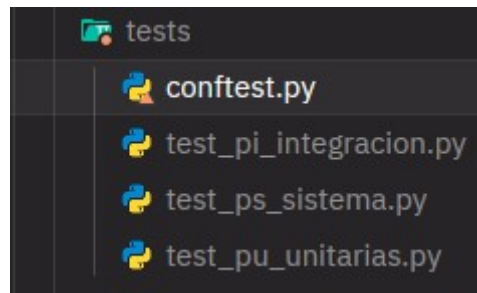
9. Implementación de Pruebas

En esta etapa del proyecto, la implementación de pruebas se ha enfocado únicamente en las funcionalidades más críticas del sistema, seleccionadas con base en los requerimientos y las historias de usuario priorizadas. Esto significa que no se están cubriendo todas las posibles funcionalidades, sino aquellas que representan mayor impacto en la operación del sistema, como el registro de ventas, la gestión de inventario, el control de acceso y la generación de reportes.

El proceso de validación se ha estructurado en tres niveles de prueba: pruebas unitarias, pruebas de integración y pruebas de sistema. Cada uno de estos niveles permite verificar el correcto funcionamiento, desde la lógica interna hasta el comportamiento completo del sistema.

Hay que aclarar que las pruebas de aceptación no se han incluido en esta fase, ya que estas requieren una validación del sistema en su totalidad bajo condiciones cercanas al uso real. Dado que el desarrollo actual se encuentra enfocado en componentes críticos y aún no se ha evaluado el sistema completo, este tipo de pruebas no serán abordadas, por lo menos en esta etapa.

9.1 Configuración del entorno de pruebas



El aseguramiento de la calidad del software requiere no solo la definición de qué se va a probar, sino también la construcción de un entorno de pruebas adecuado que permita ejecutar los casos de forma consistente, reproducible y escalable. En este contexto, el presente apartado describe la implementación del entorno de pruebas del proyecto mediante el uso del framework *pytest*.

9.1.1 Definición *pytest*

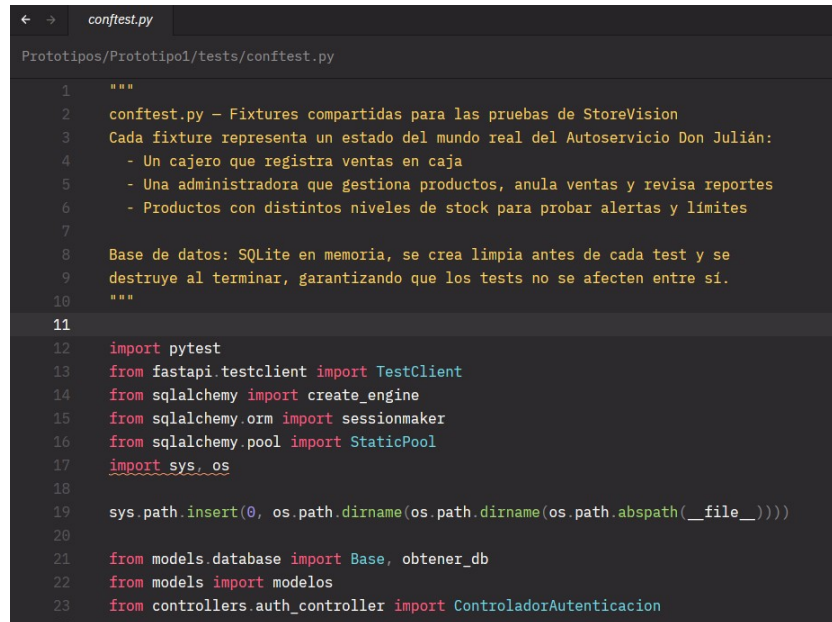
El proyecto adopta *pytest* como herramienta principal para la ejecución de pruebas automatizadas en Python. Este framework se caracteriza por su sintaxis sencilla, su flexibilidad estructural y su capacidad para manejar configuraciones complejas a través de componentes reutilizables.

Una de las principales ventajas de *pytest* es que permite escribir pruebas sin necesidad de clases obligatorias, utilizando funciones simples con aserciones directas (*assert*). Además, ofrece un sistema robusto de fixtures, el cual facilita la preparación de datos y estados previos a la ejecución de cada prueba.

En el contexto del proyecto, *pytest* permite:

- Ejecutar pruebas de manera automatizada y organizada.
- Reutilizar configuraciones comunes mediante *fixtures*.
- Garantizar independencia entre casos de prueba.
- Escalar el conjunto de pruebas a distintos niveles (unitario, integración y sistema).

9.1.2 Archivo *conftest.py* como núcleo de configuración



```

1  """
2  conftest.py – Fixtures compartidas para las pruebas de StoreVision
3  Cada fixture representa un estado del mundo real del Autoservicio Don Julián:
4  - Un cajero que registra ventas en caja
5  - Una administradora que gestiona productos, anula ventas y revisa reportes
6  - Productos con distintos niveles de stock para probar alertas y límites
7
8  Base de datos: SQLite en memoria, se crea limpia antes de cada test y se
9  destruye al terminar, garantizando que los tests no se afecten entre sí.
10 """
11
12 import pytest
13 from fastapi.testclient import TestClient
14 from sqlalchemy import create_engine
15 from sqlalchemy.orm import sessionmaker
16 from sqlalchemy.pool import StaticPool
17 import sys, os
18
19 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
20
21 from models.database import Base, obtener_db
22 from models import modelos
23 from controllers.auth_controller import ControladorAutenticacion

```

El archivo *conftest.py* constituye el eje central de la configuración del entorno de pruebas. En *pytest*, este archivo tiene un comportamiento especial: todas las *fixtures* definidas en él están disponibles automáticamente para los archivos de prueba, sin necesidad de importación explícita.

Su propósito principal es centralizar la lógica de preparación del entorno, evitando duplicación de código y asegurando consistencia en la ejecución de los tests. A través de este archivo, se definen los elementos básicos que requieren las pruebas, tales como conexiones a base de datos, usuarios de prueba y datos iniciales del sistema.

De esta manera, *conftest.py* permite desacoplar la configuración del entorno de la lógica de los tests, lo cual mejora la mantenibilidad del código y facilita su evolución.

9.1.3 Uso de fixtures en la configuración de pruebas

Las *fixtures* son funciones que proporcionan datos o estados necesarios para la ejecución de una prueba. Estas se inyectan automáticamente en los tests mediante los parámetros de las funciones, lo que simplifica la preparación del entorno.

```
@pytest.fixture(scope="function")
def db():
    """
    Sesión de base de datos limpia por cada test.
    """
    Base.metadata.create_all(bind=engine_prueba)
    sesion = SesionPrueba()
    try:
        yield sesion
    finally:
        sesion.close()
        Base.metadata.drop_all(bind=engine_prueba)
```

```
# USUARIOS - representan los roles reales del negocio

@pytest.fixture
def admin(db):
    """
    Administradora con acceso completo al sistema.
    """
    ctrl = ControladorAutenticacion(db)
    usuario = modelos.Usuario(
        email="admin@storevision.com",
        nombre="María González",
        hashed_password=ctrl.obtener_hash_password("admin123"),
        rol="administradora",
        activo=True,
    )
    db.add(usuario)
    db.commit()
    db.refresh(usuario)
    return usuario
```

Por ejemplo, una prueba puede recibir directamente una conexión a base de datos o un usuario previamente creado, sin necesidad de instanciarlo manualmente dentro del test. Este enfoque promueve la reutilización y reduce la complejidad del código de prueba.

9.1.4 Estructura de las principales fixtures

9.1.4.1 Base de datos en memoria

```
# BASE DE DATOS DE PRUEBA

engine_prueba = create_engine(
    "sqlite:/// :memory:",
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
SesionPrueba = sessionmaker(autocommit=False, autoflush=False, bind=engine_prueba)

@pytest.fixture(scope="function")
def db():
    """
    Sesión de base de datos limpia por cada test.
    """
    Base.metadata.create_all(bind=engine_prueba)
    sesion = SesionPrueba()
    try:
        yield sesion
    finally:
        sesion.close()
        Base.metadata.drop_all(bind=engine_prueba)
```

Se define una *fixture* encargada de inicializar una base de datos SQLite en memoria. Esta decisión permite ejecutar las pruebas de forma rápida y sin afectar datos reales. Además, cada prueba trabaja sobre una instancia limpia, lo que garantiza independencia y reproducibilidad.

Entrando en detalle este fixture está dividido en 7 partes principales.

1. Crea un motor de base de datos SQLite en memoria configurado para reutilizar la misma conexión durante las pruebas.

```
engine_prueba = create_engine(
    "sqlite:/// :memory:",
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
```

La base de datos de prueba solo vive durante el tiempo de ejecución de una prueba.

2. Define una fábrica de sesiones que controla cómo se gestionan las transacciones y conecta las sesiones al motor de prueba.

```
SesionPrueba = sessionmaker(autocommit=False, autoflush=False, bind=engine_prueba)
```

3. Declara un fixture de pytest con alcance por función para asegurar que cada test tenga su propio entorno aislado.

```
@pytest.fixture(scope="function")
def db():
```

4. Genera todas las tablas de la base de datos antes de ejecutar cada prueba.

```
def db():
    """
    Sesión de base de datos limpia por cada test.
    """
    Base.metadata.create_all(bind=engine_prueba)
```

5. Crea una sesión activa que permitirá interactuar con la base de datos dentro del test.

```
sesion = SesionPrueba()
```

6. Entrega la sesión al test y marca el punto donde termina la preparación e inicia la ejecución del mismo, al terminar cierra la sesión y elimina todas las tablas para dejar la base de datos limpia para la siguiente prueba.

```
try:
    yield sesion
finally:
    sesion.close()
    Base.metadata.drop_all(bind=engine_prueba)
```

9.1.4.2 Usuarios de prueba

```
@pytest.fixture
def admin(db):
    """
    Administradora con acceso completo al sistema.
    """
    ctrl = ControladorAutenticacion(db)
    usuario = modelos.Usuario(
        email="admin@storevision.com",
        nombre="Maria González",
        hashed_password=ctrl.obtener_hash_password("admin123"),
        rol="administradora",
        activo=True,
    )
    db.add(usuario)
    db.commit()
    db.refresh(usuario)
    return usuario
```

```
@pytest.fixture
def cajero(db):
    """
    Cajero con acceso limitado: solo puede registrar ventas.
    """
    ctrl = ControladorAutenticacion(db)
    usuario = modelos.Usuario(
        email="cajero@storevision.com",
        nombre="Carlos Rodríguez",
        hashed_password=ctrl.obtener_hash_password("cajero123"),
        rol="cajero",
        activo=True,
    )
    db.add(usuario)
    db.commit()
    db.refresh(usuario)
    return usuario
```

Se implementan *fixtures* que representan distintos tipos de usuarios, como administradores y cajeros. Estas permiten validar comportamientos diferenciados según permisos y roles, sin necesidad de recrear los usuarios en cada caso de prueba.

Los usuarios se crean en base al modelo definido dentro de la carpeta `models/modelos.py`

```

← → • modelos.py
Prototipos/Prototipo1/models/modelos.py

1  from sqlalchemy import Column, Integer, String, Float, DateTime, Boolean, Fo
2  from sqlalchemy.orm import relationship
3  from .database import Base
4  from datetime import datetime, timezone, timedelta
5
6  class Usuario(Base):
7      __tablename__ = "usuarios"
8
9      id = Column(Integer, primary_key=True, index=True)
10     email = Column(String(100), unique=True, index=True, nullable=False)
11     nombre = Column(String(100), nullable=False)
12     hashed_password = Column(String(255), nullable=False)
13     rol = Column(String(20), nullable=False) # administradora, cajero
14     activo = Column(Boolean, default=True)
15     fecha_creacion = Column(
16         DateTime(timezone=True),
17         default=lambda: datetime.now(timezone(timedelta(hours=-5)))
18     )

```

Y la variable `ctrl` es una instancia de la clase `ControladorAutenticacion` en el archivo `auth_controller.py` dentro de la carpeta de `controllers` cuyo unico proposito es el de encriptar la contraseña de ambos usuarios usando la funcion `obtener_hash_password`.

```

Administradora con acceso completo al sistema.
"""
ctrl = ControladorAutenticacion(db)

```

```

hashed_password=ctrl.obtener_hash_password("admin123"),

```

```

def obtener_hash_password(self, password: str):
    return pwd_context.hash(password)

```

9.1.4.3 Sesiones simuladas

```
# SESIONES HTTP - simulan un usuario ya autenticado haciendo peticiones

@pytest.fixture
def session_admin(client, admin, sucursal):
    """
    Header de sesión activa para la administradora.
    """
    r = client.post(
        "/api/login",
        json={"email": "admin@storevision.com", "password": "admin123"},
    )
    assert r.status_code == 200, "El login de admin falló al preparar la sesión"
    return {"session-id": r.json()["session_id"]}

@pytest.fixture
def session_cajero(client, cajero, sucursal):
    """
    Header de sesión activa para el cajero.
    """
    r = client.post(
        "/api/login",
        json={"email": "cajero@storevision.com", "password": "cajero123"},
    )
    assert r.status_code == 200, "El login de cajero falló al preparar la sesión"
    return {"session-id": r.json()["session_id"]}
```

```
@pytest.fixture
def session_cajero(client, cajero, sucursal):
    """
    Header de sesión activa para el cajero.
    """
    r = client.post(
        "/api/login",
        json={"email": "cajero@storevision.com", "password": "cajero123"},
    )
    assert r.status_code == 200, "El login de cajero falló al preparar la sesión"
    return {"session-id": r.json()["session_id"]}
```

Algunas *fixtures* generan sesiones autenticadas mediante encabezados o tokens simulados. Esto facilita la ejecución de pruebas a nivel de API, permitiendo replicar el comportamiento de usuarios reales sin requerir procesos completos de autenticación en cada test.

El `client.post` envía una solicitud HTTP al servidor en el endpoint `/api/login` con las credenciales para que el usuario simulado inicie sesión.

El `assert` verificara una condición, si esta es falsa detendrá la prueba y mostrara un mensaje de error, en este caso verifica si la solicitud de login fue exitosa si este devuelve un código 200.

9.1.4.4 Datos del dominio

```
@pytest.fixture
def producto(db):
    """
    Producto con stock suficiente para ventas normales.
    """
    p = modelos.Producto(
        codigo="TEST001",
        nombre="Arroz Diana 1kg",
        categoria="Granos",
        precio_venta=4500.0,
        costo=3200.0,
        stock_actual=100,
        stock_minimo=10,
        activo=True,
    )
    db.add(p)
    db.commit()
    db.refresh(p)
    return p
```

```
@pytest.fixture
def producto_en_limite(db):
    """
    Producto exactamente en el umbral mínimo (stock == mínimo).
    """
    p = modelos.Producto(
        codigo="LOW001",
        nombre="Queso Campesino 500g",
        categoria="Lácteos",
        precio_venta=12500.0,
        costo=8500.0,
        stock_actual=5,
        stock_minimo=5,
        activo=True,
    )
    db.add(p)
    db.commit()
    db.refresh(p)
    return p
```

```
@pytest.fixture
def producto_sin_stock(db):
    """
    Producto completamente agotado.
    """
    p = modelos.Producto(
        codigo="ZER001",
        nombre="Huevos AA x30",
        categoria="Lácteos",
        precio_venta=18500.0,
        costo=14500.0,
        stock_actual=0,
        stock_minimo=8,
        activo=True,
    )
    db.add(p)
    db.commit()
    db.refresh(p)
    return p
```

También se incluyen *fixtures* que representan entidades del sistema en distintos estados, como productos con stock normal, en límite mínimo o agotados. Estas configuraciones permiten cubrir tanto escenarios comunes como casos de frontera.

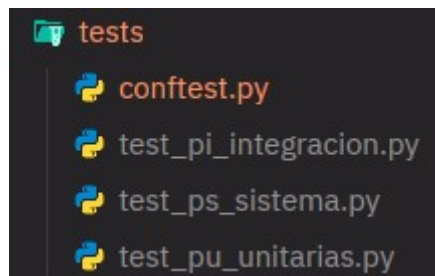
Cada uno de estos *fixtures* está basado en el modelo de productos dentro de la carpeta *models/modelos.py*

```
class Producto(Base):
    __tablename__ = "productos"

    id = Column(Integer, primary_key=True, index=True)
    codigo = Column(String(50), unique=True, index=True)
    nombre = Column(String(100), nullable=False)
    descripcion = Column(String(255))
    precio_venta = Column(Float, nullable=False)
    costo = Column(Float, nullable=False)
    stock_actual = Column(Integer, default=0)
    stock_minimo = Column(Integer, default=5)
    categoria = Column(String(50))
    activo = Column(Boolean, default=True)
```

9.1.5 Organización del conjunto de pruebas

El proyecto estructura los archivos de prueba de acuerdo con el nivel de validación:



- test_pu_unitarias.py: pruebas unitarias
- test_pi_integracion.py: pruebas de integración
- test_ps_sistema.py: pruebas de sistema

Todos estos archivos utilizan las *fixtures* definidas en `conftest.py`, lo que garantiza coherencia en la configuración y facilita la reutilización de componentes.

venta automáticamente			
US-04 Anular o corregir una venta	TestAnularVenta	TestAnulacionRestaurandoStock	TestAnulacionDeVentas
US-05 Registrar movimientos de inventario	TestMovimientosInventario	TestAtomicidadDeVentas	TestFlujoDeVenta
US-06 Actualizar inventario tras venta	TestRegistrarVenta	TestAtomicidadDeVentas	TestFlujoDeVenta
US-07 Alerta de stock mínimo	TestAlertasStock	TestAlertasTrasMovimiento	TestAlertasDeInventario
US-11 Generar balance económico	TestBalanceEconomico	—	—
US-12 Crear usuarios con roles	TestAutenticacion	TestControlDeAccesoHTTP	TestControlDeRoles
US-13 Restringir acceso por rol	TestAutenticacion	TestControlDeAccesoHTTP	TestControlDeRoles
US-22 Registrar acciones críticas (auditoría)	TestAutenticacion	TestAuditoriaDeAcceso	—

Un ejemplo de historias de uso para visualizar como están relacionadas con las pruebas es la siguiente:

Caso de usuario US-01 que describe el registro automático de ventas

STORE-1 / STORE-10

US-01 Registro automático de ventas

Tareas por hacer

Descripción

Historia de usuario

- Como cajero del Autoservicio Don Julián, cuando facturo una venta, quiero que cada venta se registre automáticamente para no duplicar información y evitar errores.

Contexto

Este issue se centra en la necesidad de automatizar el registro de ventas en el Autoservicio Don Julián. La automatización busca mejorar la eficiencia y reducir errores en el proceso de facturación.

Criterios de aceptación

- Se guarda automáticamente: producto, cantidad, precio, método de pago.
- Tiempo de respuesta < 2 segundos.
- No se pierde información en horas pico.
- Confirmación visual de registro.

Que tiene relación con las siguientes pruebas:

Prueba unitaria para el registro de una venta

```
class TestRegistrarVenta:
    def test_venta_valida_genera_id_de_venta(self, db, cajero, sucursal, producto):
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 3}],
            cajero.id,
        )
        assert "venta_id" in resultado, (
            f"La venta no retornó un ID. Respuesta recibida: {resultado}"
        )

    def test_total_es_cantidad_por_precio_unitario(self, db, cajero, sucursal, producto):
        cantidad = 3
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": cantidad}],
            cajero.id,
        )

        venta = db.query(modelos.Venta).filter_by(id=resultado["venta_id"]).first()
        total_esperado = cantidad * producto.precio_venta

        assert abs(venta.total - total_esperado) <= 0.01, (
            f"Total calculado: {venta.total}, esperado: {total_esperado}"
        )

    def test_venta_sin_productos_retorna_error(self, db, cajero, sucursal):
        resultado = ControladorVentas(db).registrar_venta(
            {"items": []},
            cajero.id,
        )

        assert "error" in resultado
```

Prueba de integración para verificar que el registro de las ventas afecta a la base de datos

```

class TestAtomicidadDeVentas:
    def test_venta_exitosa_descuenta_stock_y_crea_movimiento(self, db, cajero, sucursal, producto):
        stock_antes = producto.stock_actual
        ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 5}],
            cajero.id,
            sucursal.id
        )
        db.refresh(producto)

        # Verificar que el stock bajó
        assert producto.stock_actual == stock_antes - 5, (
            f"Stock esperado: {stock_antes - 5}, actual: {producto.stock_actual}"
        )

        # Verificar que el movimiento de inventario quedó registrado
        movimiento = db.query(modelos.MovimientoInventario).filter_by(
            producto_id=producto.id,
            tipo_movimiento="salida",
        ).first()

        assert movimiento is not None, "No se creó el movimiento de inventario"
        assert movimiento.cantidad == 5

    def test_venta_con_producto_inexistente_no_deja_registros_parciales(self, db, cajero, sucursal, producto):
        """
        Escenario: primer ítem válido, segundo ítem con ID 99999 (no existe).
        """
        ventas_antes = db.query(modelos.Venta).count()
        movimientos_antes = db.query(modelos.MovimientoInventario).count()

        ControladorVentas(db).registrar_venta(

```

Prueba de Sistema para validar que al enviar una solicitud de venta al sistema, esta se registre automáticamente de forma correcta, almacenando todos los datos requeridos y actualizando el inventario

```

class TestFlujoDeVenta:
    """
    Simula el flujo real del cajero en caja: selecciona productos,
    confirma la venta y el sistema descuenta el stock automáticamente.
    """

    def test_venta_exitosa_retorna_201(self, client, sesion_cajero, producto):
        """
        Una venta con producto disponible debe retornar HTTP 201
        (recurso creado exitosamente).
        """
        respuesta = client.post(
            "/api/ventas",
            json={"items": [{"producto_id": producto.id, "cantidad": 2}],
            headers=sesion_cajero,
        )

        assert respuesta.status_code == 201, (
            f"Se esperaba 201, se recibió {respuesta.status_code}. "
            f"Detalle: {respuesta.json()}"
        )

    def test_venta_descuenta_stock_en_base_de_datos(self, client, db, sesion_cajero, producto):
        """
        Después de confirmar la venta, el stock en la BD debe
        reflejar las unidades vendidas, sin necesidad de ninguna acción manual.
        """
        stock_antes = producto.stock_actual
        client.post(
            "/api/ventas",
            json={"items": [{"producto_id": producto.id, "cantidad": 3}],

```

9.2 Implementación Pruebas Unitarias

9.2.1 Definición pruebas unitarias

Una prueba unitaria valida el comportamiento de una unidad aislada del sistema, normalmente una función o método específico de un controlador. Este nivel de prueba se centra exclusivamente en la lógica interna de cada componente, sin interacción con módulos externos, interfaces HTTP ni flujos completos.

9.2.2 Justificación del uso de caja blanca

- La técnica de Caja Blanca es la más adecuada en el nivel unitario porque el equipo de desarrollo conoce completamente el código fuente de cada controlador. Esto permite:
- Diseñar tests que cubran todas las ramas lógicas (if/else, validaciones de stock, reglas de negocio).
- Verificar condiciones internas como la doble anulación de venta o la tolerancia numérica en totales.
- Detectar errores en la lógica antes de integrar los módulos.
- Garantizar cobertura de caminos de ejecución que de otro modo quedarían sin probar.

Diferencias con otros niveles de prueba

Aspecto	Unitario (este nivel)	Integración / E2E
Alcance	Una función/método	Módulos o flujos completos
Técnica	Caja Blanca	Caja Negra / Gris
Dependencias	Mocks / BD en memoria	Sistema real o staging
Velocidad	Muy rápida	Moderada a lenta

Aspecto	Unitario (este nivel)	Integración / E2E
Objetivo	Lógica interna	Flujo e integración
Endpoints HTTP	No se usan	Sí se usan

9.2.3 Fixtures Utilizadas en las Pruebas Unitarias

Las fixtures son funciones auxiliares de pytest que proveen el estado inicial necesario para cada prueba. Están definidas en `conftest.py` y pytest las inyecta automáticamente en cada test que las declare como parámetros. Garantizan aislamiento, reproducibilidad y resultados deterministas sin afectar datos reales.

Fixture: db — Sesión de base de datos SQLite en memoria

Esta fixture es la base de toda la infraestructura de pruebas. Crea una sesión de base de datos completamente aislada usando SQLite en memoria mediante SQLAlchemy.

Código fuente (`conftest.py`)

```
engine_prueba = create_engine(
    "sqlite:///memory:",
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
SessionPrueba = sessionmaker(autocommit=False, autoflush=False, bind=engine_prueba)

@pytest.fixture(scope="function")
def db():
    """
    Sesión de base de datos limpia por cada test.
    """
    Base.metadata.create_all(bind=engine_prueba)
    sesion = SessionPrueba()
    try:
        yield sesion
    finally:
        sesion.close()
        Base.metadata.drop_all(bind=engine_prueba)
```

Características técnicas clave

Parámetro	Propósito
sqlite:///memory:	BD completamente en RAM — no genera archivos en disco, desaparece al cerrar la sesión
StaticPool	Garantiza que todos los hilos compartan la misma conexión en memoria durante el test
check_same_thread: False	Permite que pytest acceda a la BD desde distintos contextos sin errores de threading
autocommit=False	Los cambios deben confirmarse explícitamente con db.commit(), igual que en producción
scope='function'	La BD se crea y destruye en cada función de test — aislamiento total garantizado

¿Por qué SQLite en memoria y no la BD real?

- Velocidad: la ejecución en RAM es órdenes de magnitud más rápida que en disco.
- Sin efectos secundarios: los datos de producción o desarrollo nunca son afectados.
- Reinicio automático: el bloque finally garantiza que las tablas se eliminen al terminar, dejando estado limpio para el siguiente test.
- Reproducibilidad: cada test parte exactamente del mismo punto, sin residuos de ejecuciones anteriores.

Fixtures de usuarios: admin y cajero

Representan los dos roles del negocio definidos en los User Stories US-12 y US-13. Ambas fixtures usan ControladorAutenticacion para generar el hash bcrypt de la contraseña, exactamente igual que lo haría el sistema en producción.

Código fuente — fixture admin

```

@pytest.fixture
def admin(db):
    """
    Administradora con acceso completo al sistema.
    """
    ctrl = ControladorAutenticacion(db)
    usuario = modelos.Usuario(
        email="admin@storevision.com",
        nombre="María González",
        hashed_password=ctrl.obtener_hash_password("admin123"),
        rol="administradora",
        activo=True,
    )
    db.add(usuario)
    db.commit()
    db.refresh(usuario)
    return usuario

```

Código fuente — fixture cajero

```

@pytest.fixture
def cajero(db):
    """
    Cajero con acceso limitado: solo puede registrar ventas.
    """
    ctrl = ControladorAutenticacion(db)
    usuario = modelos.Usuario(
        email="cajero@storevision.com",
        nombre="Carlos Rodríguez",
        hashed_password=ctrl.obtener_hash_password("cajero123"),
        rol="cajero",
        activo=True,
    )
    db.add(usuario)
    db.commit()
    db.refresh(usuario)
    return usuario

```

Comparativa de roles y uso en tests

Fixture	Rol en BD	Permisos que valida	Tests que la usan
admin	administradora	Crear usuarios, anular ventas, movimientos de inventario, reportes	TestAutenticacion, TestAnularVenta, TestMovimientosInventario, TestBalanceEconomico

Fixture	Rol en BD	Permisos que valida	Tests que la usan
cajero	cajero	Registrar ventas en caja	TestRegistrarVenta

La contraseña se almacena como hash bcrypt usando `obtener_hash_password()` del mismo `ControladorAutenticacion` que es objeto de prueba en `TestAutenticacion`. Esto garantiza que el test de autenticación valide el flujo real de hashing y verificación, no un atajo.

Fixture: sucursal — Contexto de tienda para ventas

El modelo `Venta` requiere una clave foránea `sucursal_id` para existir en la base de datos. El sistema `StoreVision` opera con una sola sucursal (siempre `ID = 1`), por lo que esta fixture debe cargarse antes de cualquier test que registre ventas.

Código fuente

```
@pytest.fixture
def sucursal(db):
    s = modelos.Sucursal(nombre="Tienda StoreVision", direccion="Cra 15 #45-60")
    db.add(s)
    db.commit()
    db.refresh(s)
    return s
```

¿Por qué es necesaria?

- La tabla `Venta` tiene una restricción de integridad referencial: `sucursal_id` NOT NULL con FK a `Sucursal`.
- `ControladorVentas.registrar_venta()` asigna `sucursal_id = 1` de forma fija; si la sucursal no existe en BD, la inserción falla con error de FK.
- Al ser una fixture de función, la sucursal se crea nueva en cada test, manteniendo el aislamiento.

Fixtures de productos — Particiones de equivalencia del inventario

Los tres fixtures de productos cubren sistemáticamente los estados posibles del inventario según los User Stories US-06 y US-07. Cada uno activa una partición de equivalencia distinta, lo que permite probar el comportamiento correcto del sistema ante cada escenario sin repetir código.

producto — Stock suficiente (partición válida)

```
@pytest.fixture
def producto(db):
    """
    Producto con stock suficiente para ventas normales.
    """
    p = modelos.Producto(
        codigo="TEST001",
        nombre="Arroz Diana 1kg",
        categoria="Granos",
        precio_venta=4500.0,
        costo=3200.0,
        stock_actual=100,
        stock_minimo=10,
        activo=True,
    )
    db.add(p)
    db.commit()
    db.refresh(p)
    return p
```

Atributo	Valor y significado
stock_actual	100 unidades — muy por encima del mínimo, habilita ventas y movimientos sin restricción
stock_minimo	10 — umbral que activa alertas; con stock=100 NO debe generar alerta
precio_venta / costo	\$4.500 / \$3.200 — diferencia de \$1.300 usada para calcular utilidad bruta en TestBalanceEconomico
Tests que la usan	TestRegistrarVenta, TestMovimientosInventario, TestAlertasStock,

Atributo	Valor y significado
	TestBalanceEconomico

producto_sin_stock — Stock agotado (caso extremo, stock = 0)

```
@pytest.fixture
def producto_sin_stock(db):
    """
    Producto completamente agotado.
    """
    p = modelos.Producto(
        codigo="ZERO01",
        nombre="Huevos AA x30",
        categoria="Lácteos",
        precio_venta=18500.0,
        costo=14500.0,
        stock_actual=0,
        stock_minimo=8,
        activo=True,
    )
    db.add(p)
    db.commit()
    db.refresh(p)
    return p
```

Atributo	Valor y significado
stock_actual	0 — producto completamente agotado, caso extremo del inventario
stock_minimo	8 — la condición $0 \leq 8$ activa la alerta de inventario
Comportamiento	Venta rechazada con error + producto aparece en lista de alertas
Tests que la usan	TestRegistrarVenta (agotado), TestAlertasStock (alerta por stock=0)

producto_en_limite — Stock en umbral mínimo (caso de frontera inclusivo)

```

@pytest.fixture
def producto_en_limite(db):
    """
    Producto exactamente en el umbral mínimo (stock == mínimo).
    """
    p = modelos.Producto(
        codigo="LOW001",
        nombre="Queso Campesino 500g",
        categoria="Lácteos",
        precio_venta=12500.0,
        costo=8500.0,
        stock_actual=5,
        stock_minimo=5,
        activo=True,
    )
    db.add(p)
    db.commit()
    db.refresh(p)
    return p

```

Atributo	Valor y significado
stock_actual	5 — exactamente igual al mínimo, no por debajo
stock_minimo	5 — la condición stock_actual <= stock_minimo es verdadera por igualdad (frontera inclusiva)
Caso de frontera	Valida que el operador <= del controlador sea inclusivo; si fuera < el test fallaría
Tests que la usan	TestAlertasStock (test_producto_en_limite_minimo_genera_alerta)

9.2.4 Principio de aislamiento por test

El diseño de scope='function' en la fixture db garantiza que cada prueba tenga su propio universo de datos. El flujo completo es:

Fase	Acción	Efecto
1	Base.metadata.create_all()	Crea todas las tablas del modelo en SQLite en memoria
2	SesionPrueba()	Abre una sesión transaccional nueva
3	yield sesion	Ejecuta el test; las fixtures de datos (admin, producto...) insertan sus registros
4	sesion.close()	Cierra la conexión y libera recursos
5	Base.metadata.drop_all()	Elimina todas las tablas — la BD queda vacía para el siguiente test

- Independencia: el fallo o éxito de un test no afecta el estado que verá el siguiente.
- Reproducibilidad: ejecutar la suite 10 veces produce exactamente los mismos resultados.
- Sin limpieza manual: no es necesario escribir código teardown en cada test; la fixture lo gestiona automáticamente.

9.2.5 Análisis de Clases de Prueba

A continuación, se documenta en detalle cada clase de prueba del archivo `test_pu_unitarias.py`, su relación con el controlador correspondiente y la cobertura que aporta.

9.2.5.1 Clase: TestAutenticacion

```
class TestAutenticacion:
    def test_credenciales_correctas_retornan_usuario(self, db, admin):
        resultado = ControladorAutenticacion(db).autenticar_usuario(
            "admin@storevision.com", "admin123"
        )

        assert resultado is not None
        assert resultado.rol == "administradora"

    def test_password_incorrecta_bloquea_acceso(self, db, admin):
        resultado = ControladorAutenticacion(db).autenticar_usuario(
            "admin@storevision.com", "password_incorrecta"
        )

        assert resultado is None

    def test_login_fallido_queda_registrado_en_auditoria(self, db, admin):
        ControladorAutenticacion(db).autenticar_usuario(
            "admin@storevision.com", "password_incorrecta"
        )

        registro = db.query(modelos.RegistroAuditoria).filter_by(
            tipo_accion="login_fallido"
        ).first()

        assert registro is not None, (
            "El intento fallido de login no quedó registrado en auditoría"
        )
```

Justificación:

El sistema maneja datos sensibles y acceso por roles. Sin una autenticación correcta, cualquier usuario podría entrar con credenciales ajenas, crearse con email duplicado o actuar sin dejar rastro. Esta clase garantiza que la puerta de entrada al sistema funciona exactamente como debe: dejando pasar solo a quien corresponde, bloqueando al resto y registrando todo intento fallido.

Controlador: auth_controller.py

El ControladorAutenticacion gestiona autenticación con bcrypt (passlib) y registra en RegistroAuditoria cada intento exitoso o fallido. La lógica interna relevante para caja blanca es:

Rama 1: usuario no encontrado o contraseña incorrecta → registra login_fallido en auditoría, retorna None.

Rama 2: usuario encontrado, password válida, activo=True → registra login_exitoso, retorna objeto Usuario.

Rama 3: email ya registrado en crear_usuario → retorna {'error': '...'} sin duplicar.

Detalle de cada test

Test	Tipo de partición	Verificación
test_credenciales_correctas_retornan_usuario	Partición válida	resultado.rol == 'administradora'
test_password_incorrecta_bloquea_acceso	Partición inválida	resultado is None
test_login_fallido_queda_registrado_en_auditoria	Efecto secundario	RegistroAuditoria con tipo_accion='login_fallido'
test_email_duplicado_no_crea_usuario	Regla de unicidad	Respuesta con 'error', count == 1

9.2.5.2 Clase: *TestRegistrarVenta*

```
class TestRegistrarVenta:
    def test_venta_valida_genera_id_venta(self, db, cajero, sucursal, producto):
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 3}]},
            cajero.id,
        )
        assert "venta_id" in resultado, (
            f"La venta no retornó un ID. Respuesta recibida: {resultado}"
        )

    def test_total_es_cantidad_por_precio_unitario(self, db, cajero, sucursal, producto):
        cantidad = 3
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": cantidad}]},
            cajero.id,
        )

        venta = db.query(modelos.Venta).filter_by(id=resultado["venta_id"]).first()
        total_esperado = cantidad * producto.precio_venta

        assert abs(venta.total - total_esperado) <= 0.01, (
            f"Total calculado: {venta.total}, esperado: {total_esperado}"
        )

    def test_venta_sin_productos_retorna_error(self, db, cajero, sucursal):
        resultado = ControladorVentas(db).registrar_venta(
            {"items": []},
            cajero.id,
        )
```

```
        assert "error" in resultado

    def test_venta_con_stock_insuficiente_no_modifica_inventario(self, db, cajero, sucursal, producto):
        stock_antes = producto.stock_actual
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 9999}]},
            cajero.id,
        )
        db.refresh(producto)

        assert "error" in resultado
        assert producto.stock_actual == stock_antes, (
            "El stock cambió aunque la venta debió rechazarse por insuficiencia"
        )

    def test_venta_con_producto_agotado_retorna_error(self, db, cajero, sucursal, producto_sin_stock):
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto_sin_stock.id, "cantidad": 1}]},
            cajero.id,
        )

        assert "error" in resultado
```

Justificación:

El registro de ventas es la operación más frecuente del negocio y la que más consecuencias tiene si falla: afecta el inventario, el dinero y los reportes simultáneamente. Esta clase cubre desde el flujo exitoso hasta los rechazos por datos inválidos, garantizando que ninguna venta mal formada, con stock insuficiente o con producto agotado llegue a modificar la base de datos.

Controlador: `ventas_controller.py`

El `ControladorVentas.registrar_venta()` ejecuta la siguiente lógica interna que es cubierta por caja blanca:

- Validación de items no vacíos — retorna error si la lista está vacía.
- Por cada producto: consulta stock, valida suficiencia, calcula subtotal.
- Si el stock es insuficiente → retorna error SIN modificar la BD (rollback implícito).
- Si todo es válido: crea Venta, ItemVenta, actualiza stock (salida), registra MovimientoInventario y auditoría.
- Caso de frontera: el total debe coincidir con $\text{cantidad} \times \text{precio_venta}$ con tolerancia ± 0.01 (punto flotante).

Detalle de cada test

Test	Caso	Verificación clave
test_venta_valida_genera_id_de_venta	Flujo principal	'venta_id' en resultado
test_total_es_cantidad_por_precio_unitario	Frontera ± 0.01	$\text{abs}(\text{venta.total} - \text{esperado}) \leq 0.01$
test_venta_sin_productos_retorna_error	Entrada vacía	'error' en resultado
test_venta_con_stock_insuficiente_no_modifica_inventario	Stock extremo (9999)	stock no cambia + error
test_venta_con_producto_agotado_retorna_error	Stock = 0	'error' en resultado

9.2.5.3 Clase: *TestAnularVenta*

```
class TestAnularVenta:
    def test_no_se_puede_anular_una_venta_ya_anulada(self, db, admin, sucursal, producto):
        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 2}]},
            admin.id,
        )
        venta_id = resultado["venta_id"]

        ctrl = ControladorVentas(db)
        ctrl.anular_venta(venta_id, admin.id, "Error en caja - primera vez")
        segunda_anulacion = ctrl.anular_venta(venta_id, admin.id, "Intento duplicado")

        assert "error" in segunda_anulacion, (
            "El sistema permitió anular dos veces la misma venta"
        )
```

Justificación:

Anular una venta restituye stock al inventario. Si esa operación se puede ejecutar dos veces sobre la misma venta, el sistema estaría creando unidades inexistentes. Esta clase protege la integridad del inventario verificando que una venta ya anulada quede bloqueada para cualquier intento posterior.

Controlador: *ventas_controller.py*

La función `anular_venta()` valida primero que la venta existe, luego que su estado no sea 'anulada'. Si ya fue anulada, retorna error. La cobertura de caja blanca relevante es la rama:

```
if venta.estado == "anulada":
    return {"error": "La venta ya está anulada"}
```

El test ejecuta una venta válida, la anula una primera vez (debe retornar éxito), y luego la anula de nuevo. El segundo intento debe retornar 'error', garantizando que el stock no se restituya dos veces.

9.2.5.4 Clase: *TestMovimientosInventario*

```
class TestMovimientosInventario:
    def test_entrada_de_mercancia_incrementa_el_stock(self, db, admin, producto):
        stock_antes = producto.stock_actual
        cantidad_recibida = 20

        ControladorInventario(db).registrar_movimiento(
            {
                "producto_id": producto.id,
                "tipo_movimiento": "entrada",
                "cantidad": cantidad_recibida,
                "motivo": "Compra a proveedor",
            },
            admin.id,
        )
        db.refresh(producto)

        assert producto.stock_actual == stock_antes + cantidad_recibida, (
            f"Stock esperado: {stock_antes + cantidad_recibida}, "
            f"stock actual: {producto.stock_actual}"
        )

    def test_salida_mayor_al_stock_retorna_error(self, db, admin, producto):
        resultado = ControladorInventario(db).registrar_movimiento(
            {
                "producto_id": producto.id,
                "tipo_movimiento": "salida",
                "cantidad": 9999,
                "motivo": "Ajuste",
            },
            admin.id,
        )

        assert "error" in resultado
```

Justificación:

El inventario es el activo central del negocio. Esta clase valida las dos únicas operaciones que modifican el stock directamente: que una entrada sume correctamente y que una salida que supere el stock disponible sea rechazada antes de producir valores negativos o inconsistentes.

Controlador: *inventario_controller.py*

La función `registrar_movimiento()` maneja dos ramas lógicas principales:

- Entrada: $\text{stock_nuevo} = \text{stock_anterior} + \text{cantidad}$. Siempre permitida.
- Salida: verifica $\text{stock_anterior} \geq \text{cantidad}$. Si no cumple, retorna error sin modificar BD.

Test	Tipo	Verificación
test_entrada_de_mercancia_incrementa_el_stock	Partición válida	stock_actual == stock_antes + 20

Test	Tipo	Verificación
test_salida_mayor_al_stock_retorna_error	Límite superior (9999)	'error' en resultado

9.2.5.5 Clase: TestAlertasStock

```
class TestAlertasStock:
    def test_producto_en_limite_minimo_genera_alerta(self, db, producto_en_limite):
        alertas = ControladorInventario(db).verificar_alertas_inventario()
        ids_en_alerta = [a["producto_id"] for a in alertas]

        assert producto_en_limite.id in ids_en_alerta, (
            "Un producto con stock igual al mínimo no aparece en alertas, "
            "pero debería (condición inclusiva stock <= mínimo)"
        )

    def test_producto_con_stock_suficiente_no_genera_alerta(self, db, producto):
        alertas = ControladorInventario(db).verificar_alertas_inventario()
        ids_en_alerta = [a["producto_id"] for a in alertas]

        assert producto.id not in ids_en_alerta, (
            "Un producto con stock normal apareció en alertas (falsa alarma)"
        )

    def test_producto_agotado_tambien_genera_alerta(self, db, producto_sin_stock):
        alertas = ControladorInventario(db).verificar_alertas_inventario()
        ids_en_alerta = [a["producto_id"] for a in alertas]

        assert producto_sin_stock.id in ids_en_alerta, (
            "Un producto agotado (stock=0) no aparece en alertas"
        )
```

Justificación:

Un sistema de alertas que falla silenciosamente es peor que no tenerlo. Esta clase verifica los tres escenarios que el sistema debe discriminar correctamente: alertar cuando el stock toca el mínimo, alertar cuando está agotado, y no alertar cuando el stock es suficiente. Los tres juntos garantizan que la condición $\leq$ del controlador funciona sin falsos positivos ni falsos negativos.

Controlador: inventario_controller.py

La función `verifica_alertas_inventario()` ejecuta la consulta:

```
Producto.stock_actual <= Producto.stock_minimo
```

El operador $\leq$ hace que la condición sea inclusiva: un producto con stock exactamente igual al mínimo DEBE generar alerta. Este es el caso de frontera crítico documentado.

Test	Condición	Resultado esperado
test_producto_en_limite_minimo_genera_alerta	$\text{stock} == \text{stock_minimo}$ (frontera $\leq$)	producto_id EN alertas
test_producto_con_stock_suficiente_no_genera_alerta	$\text{stock} >> \text{stock_minimo}$	producto_id NO en alertas
test_producto_agotado_tambien_genera_alerta	$\text{stock} = 0$	producto_id EN alertas

9.2.5.6 Clase: TestBalanceEconomico

```
class TestBalanceEconomico:
    def test_utilidad_bruta_es_ventas_menos_costos(self, db, admin, sucursal, producto):
        """
        US-11: Utilidad bruta = total de ventas - costo de ventas.
        Esta es la métrica financiera fundamental del balance.
        """
        ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 2}]},
            admin.id,
        )

        balance = ControladorReportes(db).generar_balance_economico(
            datetime(2024, 1, 1),
            datetime(2029, 12, 31),
        )

        utilidad_calculada = balance["rentabilidad"]["utilidad_bruta"]
        utilidad_esperada = (
            balance["resumen_ventas"]["total_ventas"]
            - balance["rentabilidad"]["costo_ventas"]
        )

        assert abs(utilidad_calculada - utilidad_esperada) <= 0.01, (
            f"Utilidad calculada: {utilidad_calculada}, "
            f"esperada (ventas - costos): {utilidad_esperada}"
        )
```

Justificación:

Los reportes financieros son la base para decisiones del negocio. Un error en la fórmula $\text{utilidad_bruta} = \text{ventas} - \text{costos}$ no produce un crash visible, simplemente devuelve un número incorrecto que nadie nota hasta que ya causó daño. Esta clase verifica que el cálculo es matemáticamente correcto contra una venta de referencia controlada.

Controlador: reportes_controller.py

La función `generar_balance_economico()` calcula la utilidad bruta aplicando la fórmula financiera fundamental:

```
utilidad_bruta = total_ventas - costo_ventas
```

El test registra una venta de prueba y luego verifica que la fórmula del balance produce el valor correcto. La tolerancia ± 0.01 cubre errores de punto flotante en operaciones con decimales. Esta validación cubre el User Story US-11 del sistema.

Campo verificado	Expresión del test
utilidad_calculada	balance['rentabilidad']['utilidad_bruta']
utilidad_esperada	total_ventas - costo_ventas
Tolerancia aceptada	abs(calculada - esperada) <= 0.01

9.2.5 Rol de los controladores

Controladores cubiertos por las pruebas unitarias
• auth_controller.py — Autenticación y gestión de usuarios
• ventas_controller.py — Registro y anulación de ventas
• inventario_controller.py — Movimientos y alertas de inventario
• reportes_controller.py — Balance económico y reportes financieros

9.2.6 Casos de Frontera Documentados

Los casos de frontera son condiciones límite que separan el comportamiento válido del inválido. Son especialmente importantes en caja blanca porque se diseñan conociendo las condiciones exactas del código.

#	Caso de Frontera	Condición en código	Clase de prueba	Tipo
1	Stock igual al mínimo genera alerta	stock_actual <= stock_minimo (inclusivo)	TestAlertasStock	Inclusivo
2	Producto completamente agotado	stock_actual = 0	TestAlertasStock / TestRegistrarVenta	Extremo

#	Caso de Frontera	Condición en código	Clase de prueba	Tipo
3	Total de venta con flotantes	abs(total - esperado) <= 0.01	TestRegistrarVenta	Tolerancia
4	Doble anulación de venta	venta.estado == 'anulada'	TestAnularVenta	Restricción
5	Cantidad extrema en salida (9999)	stock < cantidad solicitada	TestMovimientosInventari o	Límite superior
6	Utilidad bruta con tolerancia	abs(calculada - esperada) <= 0.01	TestBalanceEconomico	Tolerancia

9.2.7 Resultados de Ejecución

El comando para ejecutar las pruebas unitarias con salida detallada es:

```
pytest tests/test_pu_unitarias.py
```

Con un resultado de:

```
(.venv) PS C:\Users\samue\ING_SOFTWARE_III\Prototipos\Prototipo1> pytest tests/test_pu_unitarias.py
platform win32 -- Python 3.12.10, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\samue\ING_SOFTWARE_III\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\samue\ING_SOFTWARE_III\Prototipos\Prototipo1
configfile: pytest.ini
plugins: anyio-4.12.1
collected 16 items

tests/test_pu_unitarias.py::TestAutenticacion::test_credenciales_correctas_retornan_usuario PASSED [ 6%]
tests/test_pu_unitarias.py::TestAutenticacion::test_password_incorrecta_bloquea_acceso PASSED [ 12%]
tests/test_pu_unitarias.py::TestAutenticacion::test_login_fallido_queda_registrado_en_auditoria PASSED [ 18%]
tests/test_pu_unitarias.py::TestAutenticacion::test_email_duplicado_no_crea_usuario PASSED [ 25%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_valida_genera_id_de_venta PASSED [ 31%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_total_es_cantidad_por_precio_unitario PASSED [ 37%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_sin_productos_retorna_error PASSED [ 43%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_con_stock_insuficiente_no_modifica_inventario PASSED [ 50%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_con_producto_agotado_retorna_error PASSED [ 56%]
tests/test_pu_unitarias.py::TestAnularVenta::test_no_se_puede_anular_una_venta_ya_anulada PASSED [ 62%]
tests/test_pu_unitarias.py::TestMovimientosInventario::test_entrada_de_mercancia_incrementa_el_stock PASSED [ 68%]
tests/test_pu_unitarias.py::TestMovimientosInventario::test_salida_mayor_al_stock_retorna_error PASSED [ 75%]
tests/test_pu_unitarias.py::TestAlertasStock::test_producto_en_limite_minimo_genera_alerta PASSED [ 81%]
tests/test_pu_unitarias.py::TestAlertasStock::test_producto_con_stock_suficiente_no_genera_alerta PASSED [ 87%]
tests/test_pu_unitarias.py::TestAlertasStock::test_producto_agotado_tambien_genera_alerta PASSED [ 93%]
tests/test_pu_unitarias.py::TestBalanceEconomico::test_utilidad_bruta_es_ventas_menos_costos PASSED [100%]

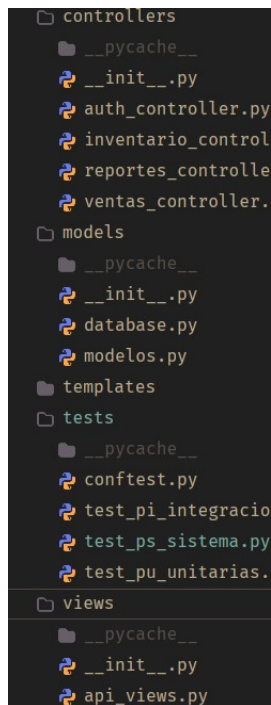
===== warnings summary =====
models/database.py:11
  C:\Users\samue\ING_SOFTWARE_III\Prototipos\Prototipo1\models\database.py:11: MovedIn20Warning: The ``declarative_base()``
  function is now available as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at:
  https://sqlalche.me/e/b8d9)
    Base = declarative_base()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 16 passed, 1 warning in 3.44s =====
```

Este conjunto de pruebas unitarias constituye la primera línea de defensa en la calidad del software, detectando errores en la lógica individual de cada componente antes de que sean integrados en flujos completos. Su correcta ejecución (todos en PASSED) es prerequisite para avanzar a las pruebas de integración y sistema.

9.3 Implementación Pruebas de Integración

9.3.1 Definición de prueba de integración



Las pruebas de integración verifican la interacción correcta entre múltiples componentes del sistema. A diferencia de las pruebas unitarias, que evalúan funciones aisladas sin dependencias externas, las pruebas de integración ejercitan flujos completos en los que intervienen dos o más capas: controladores, modelos, base de datos y endpoints HTTP. El objetivo es detectar fallos que no se manifiestan cuando los componentes se prueban de forma individual, como inconsistencias en transacciones, errores de persistencia o conflictos en las reglas de acceso.

En StoreVision, este nivel de prueba verifica que, por ejemplo, registrar una venta no solo llame correctamente al controlador sino que también descuente el stock en la base de datos, registre el movimiento de inventario y deje trazabilidad en la tabla de auditoría, todo dentro de una misma transacción atómica.

9.3.2 Justificación del uso de caja gris

La técnica de caja gris combina elementos de la caja blanca y la caja negra. El evaluador conoce parcialmente la lógica interna del sistema (estructura de los controladores, esquema de la base de datos, reglas de transacción), pero las validaciones se realizan a través de flujos completos y no sobre funciones aisladas. Esta técnica es la más adecuada para el nivel de integración por tres razones:

- Permite diseñar escenarios de fallo significativos. El conocimiento de la estructura interna habilita la construcción de casos que ejerciten rutas críticas, como transacciones con ítems mixtos o cruces de umbrales de stock mínimo.
- Habilita la validación de efectos secundarios. Se verifican no solo las respuestas visibles (retorno de función o código HTTP), sino también el estado persistido en la base de datos, lo que es imposible desde la caja negra pura.
- Establece un punto intermedio entre aislamiento y realismo. Las pruebas de caja gris son más completas que las unitarias, que no tocan la base de datos, y más controladas que las de sistema, que actúan como usuarios finales sin conocimiento interno.

La diferencia fundamental entre los tres niveles de prueba aplicados en StoreVision se resume en la siguiente tabla.

Característica	Unitaria	Integración	Sistema
Técnica	Caja blanca	Caja gris	Caja negra
Alcance	Función aislada	Capas interactuando	Flujo HTTP completo
BD real	No	Sí (en memoria)	Sí (en memoria)
Conocimiento interno	Total	Parcial	Ninguno

Tabla 1

Nota. Comparativa entre los tres niveles de prueba aplicados en el proyecto StoreVision.

Fixtures Utilizadas en las Pruebas de Integración

Las fixtures son definidas en `confest.py` y representan el contexto reutilizable que todos los tests comparten. En las pruebas de integración, a diferencia de las unitarias, las fixtures también levantan el cliente HTTP y crean sesiones de usuario autenticadas, acercándose al comportamiento real del sistema.

Base de datos en memoria

La fixture `db` crea un motor SQLite con `StaticPool`, lo que garantiza que todas las operaciones de una misma sesión de prueba compartan la misma conexión. Antes de cada test se crean todas las tablas con `Base.metadata.create_all()` y al finalizar se destruyen con `drop_all()`. Este mecanismo produce aislamiento total entre tests: ningún dato de un caso contamina el siguiente.

Cliente HTTP

La fixture `client` crea una instancia de `TestClient` de FastAPI que apunta a una aplicación de prueba. Esto permite enviar peticiones HTTP reales sin levantar un servidor Uvicorn, obteniendo respuestas completas (código de estado y cuerpo JSON) que reflejan el comportamiento exacto de la aplicación en producción.

Usuarios y sesiones autenticadas

Las fixtures `admin` y `cajero` insertan usuarios directamente en la base de datos con contraseñas hashadas mediante `bcrypt`. Las fixtures `sesion_admin` y `sesion_cajero` realizan un login real vía `POST /api/login` y capturan el `session-id` retornado, que luego se incluye como header en las peticiones a endpoints protegidos. Este mecanismo garantiza que el control de acceso sea evaluado con el mismo flujo de autenticación que usa la aplicación en producción.

Producto y sucursal

La fixture producto provee un objeto Producto con stock_actual=100 y stock_minimo=10, valores suficientes para ejecutar ventas sin interferencia de alertas. La fixture sucursal crea la entidad requerida como clave foránea en cada Venta. Ambas se insertan directamente en la base de datos, no a través de la API, lo que constituye el conocimiento interno propio del nivel caja gris.

9.3.3 Análisis de Cada Clase de Prueba

9.3.3.1 TestAuditoriaDeAcceso

Esta clase verifica que las acciones críticas de autenticación queden registradas en la tabla RegistroAuditoria, garantizando trazabilidad y cumplimiento normativo del sistema.

```
class TestAuditoriaDeAcceso:
    def test_login_exitoso_queda_en_tabla_auditoria(self, db, admin):
        ControladorAutenticacion(db).autenticar_usuario(
            "admin@storevision.com", "admin123"
        )

        registro = db.query(modelos.RegistroAuditoria).filter_by(
            usuario_id=admin.id,
            tipo_accion="login_exitoso",
        ).first()

        assert registro is not None, (
            "El login exitoso no dejó rastro en la tabla de auditoría"
        )
```

test_login_exitoso_queda_en_tabla_auditoria.

```

class TestAuditoriaDeAcceso:
    def test_login_exitoso_queda_en_tabla_auditoria(self, db, admin):
        ControladorAutenticacion(db).autenticar_usuario(
            "admin@storevision.com", "admin123"
        )

        registro = db.query(modelos.RegistroAuditoria).filter_by(
            usuario_id=admin.id,
            tipo_accion="login_exitoso",
        ).first()

        assert registro is not None, (
            "El login exitoso no dejó rastro en la tabla de auditoría"
        )

```

Este caso invoca directamente el controlador de autenticación con credenciales válidas del usuario administrador y luego consulta la tabla RegistroAuditoria para confirmar que existe un registro de tipo login_exitoso vinculado al identificador del usuario autenticado. La validación se realiza a nivel de base de datos y no de respuesta HTTP, lo que distingue a este test del nivel unitario, donde la verificación se limita al valor de retorno de la función.

El riesgo de no contar con esta prueba es que el sistema podría autenticar usuarios sin generar ningún rastro auditable, lo que impediría investigar accesos no autorizados o rastrear responsabilidades en caso de incidentes de seguridad. La Tabla 2 presenta la ficha completa de este caso.

Atributo	Detalle
Identificador	test_login_exitoso_queda_en_tabla_auditoria
Clase de prueba	TestAuditoriaDeAcceso
Función ejercitada	ControladorAutenticacion.autenticar_usuario()
Tipo de validación	Persistencia y seguridad
Partición de equivalencia	Credenciales válidas (partición positiva)
Resultado esperado	Registro != None con tipo_accion == 'login_exitoso'
Resultado obtenido	PASA

Tabla 2

Nota. Ficha de la prueba test_login_exitoso_queda_en_tabla_auditoria.

9.3.3.2 TestAtomicidadDeVentas

Esta clase verifica que el proceso de registro de ventas sea atómico: si todos los ítems son válidos, la venta se completa con todos sus efectos secundarios; si algún ítem falla, ningún dato queda persistido en la base de datos. La clase contiene dos casos complementarios.

```
class TestAtomicidadDeVentas:
    def test_venta_exitosa_descuenta_stock_y_crea_movimiento(self, db, cajero, sucursal, producto):
        stock_antes = producto.stock_actual
        ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 5}]},
            cajero.id,
        )
        db.refresh(producto)

        # Verificar que el stock bajó
        assert producto.stock_actual == stock_antes - 5, (
            f"Stock esperado: {stock_antes - 5}, actual: {producto.stock_actual}"
        )

        # Verificar que el movimiento de inventario quedó registrado
        movimiento = db.query(modelos.MovimientoInventario).filter_by(
            producto_id=producto.id,
            tipo_movimiento="salida",
        ).first()

        assert movimiento is not None, "No se creó el movimiento de inventario"
        assert movimiento.cantidad == 5

    def test_venta_con_producto_inexistente_no_deja_registros_parciales(self, db, cajero, sucursal, producto):
        """
        Escenario: primer ítem válido, segundo ítem con ID 99999 (no existe).
        """
        ventas_antes = db.query(modelos.Venta).count()
        movimientos_antes = db.query(modelos.MovimientoInventario).count()

        ControladorVentas(db).registrar_venta(
            {
                "items": [
                    {"producto_id": producto.id, "cantidad": 1},  # válido
                    {"producto_id": 99999, "cantidad": 1},        # no existe → error
                ]
            },
            cajero.id,
        )

        assert db.query(modelos.Venta).count() == ventas_antes, (
            "Se creó una Venta aunque la transacción debió revertirse"
        )
        assert db.query(modelos.MovimientoInventario).count() == movimientos_antes, (
            "Se crearon movimientos de inventario aunque la transacción debió revertirse"
        )
```

test_venta_exitosa_descuenta_stock_y_crea_movimiento.

```
def test_venta_exitosa_descuenta_stock_y_crea_movimiento(self, db, cajero, sucursal, producto):
    stock_antes = producto.stock_actual
    ControladorVentas(db).registrar_venta(
        {"items": [{"producto_id": producto.id, "cantidad": 5}]},
        cajero.id,
    )
    db.refresh(producto)

    # Verificar que el stock bajó
    assert producto.stock_actual == stock_antes - 5, (
        f"Stock esperado: {stock_antes - 5}, actual: {producto.stock_actual}"
    )

    # Verificar que el movimiento de inventario quedó registrado
    movimiento = db.query(modelos.MovimientoInventario).filter_by(
        producto_id=producto.id,
        tipo_movimiento="salida",
    ).first()

    assert movimiento is not None, "No se creó el movimiento de inventario"
    assert movimiento.cantidad == 5
```

Este caso registra una venta de cinco unidades de un producto con stock suficiente y valida dos efectos en la base de datos: que el stock del producto se haya reducido en exactamente cinco unidades, y que exista un MovimientoInventario de tipo salida con cantidad cinco asociado a ese producto. La prueba confirma la consistencia entre el módulo de ventas y el módulo de inventario, dos capas que en las pruebas unitarias se evalúan de forma independiente.

El riesgo de no contar con esta prueba es que podrían existir inconsistencias entre el stock lógico registrado en la base de datos y el stock físico real de la tienda, generando discrepancias difíciles de detectar manualmente.

test_venta_con_producto_inexistente_no_deja_registros_parciales.

```
def test_venta_con_producto_inexistente_no_deja_registros_parciales(self, db, cajero, sucursal, producto):
    """
    Escenario: primer ítem válido, segundo ítem con ID 99999 (no existe).
    """
    ventas_antes = db.query(modelos.Venta).count()
    movimientos_antes = db.query(modelos.MovimientoInventario).count()

    ControladorVentas(db).registrar_venta(
        {
            "items": [
                {"producto_id": producto.id, "cantidad": 1}, # válido
                {"producto_id": 99999, "cantidad": 1},        # no existe + error
            ],
        },
        cajero.id,
    )

    assert db.query(modelos.Venta).count() == ventas_antes, (
        "Se creó una Venta aunque la transacción debió revertirse"
    )
    assert db.query(modelos.MovimientoInventario).count() == movimientos_antes, (
        "Se crearon movimientos de inventario aunque la transacción debió revertirse"
    )
```

Este caso construye una venta con dos ítems: el primero apunta a un producto existente en la base de datos, el segundo a un producto con identificador 99999 que no existe. El test verifica que, tras la ejecución, el conteo de objetos Venta y MovimientoInventario sea idéntico al conteo previo a la llamada.

Este escenario ejercita el principio de atomicidad: una transacción debe completarse en su totalidad o no dejar ningún efecto. Sin rollback explícito, el primer ítem válido podría haber descontado stock y creado un movimiento, dejando la base de datos en un estado inconsistente. El riesgo de no validarlo es la corrupción silenciosa de datos. La Tabla 3 compara ambos casos de la clase.

Atributo	Test 1	Test 2
Escenario	Todos los ítems válidos	Ítem válido + ítem inexistente
Tipo de validación	Consistencia entre módulos	Atomicidad transaccional
Verificación en BD	stock -= 5 y MovimientoInventario creado	0 Ventas y 0 Movimientos nuevos
Resultado	PASA	PASA

Tabla 3

Nota. Comparativa de los dos casos de la clase TestAtomicidadDeVentas.

9.3.3.3 TestAnulacionRestaurandoStock

Esta clase verifica que el proceso de anulación de una venta revierta exactamente los efectos que esa venta produjo sobre el inventario, sin agregar ni restar unidades adicionales.

```

class TestAnulacionRestaurandoStock:
    """
    Verifica que anular una venta devuelve el stock al valor original,
    sin más ni menos unidades de las que se habían descontado.

    Contexto del negocio: Una anulación incorrecta (ej: restaura el doble)
    generaría un inventario inflado que no corresponde a la realidad física.
    """

    def test_anular_venta_restaura_stock_al_valor_original(
        self, db, admin, sucursal, producto
    ):
        """
        US-04: Después de registrar una venta y luego anularla,
        el stock debe quedar exactamente igual que al principio.
        """
        stock_original = producto.stock_actual

        resultado = ControladorVentas(db).registrar_venta(
            {"items": [{"producto_id": producto.id, "cantidad": 5}]},
            admin.id,
        )
        ControladorVentas(db).anular_venta(
            resultado["venta_id"], admin.id, "Error en digitación"
        )
        db.refresh(producto)

        assert producto.stock_actual == stock_original, (
            f"Stock después de anular: {producto.stock_actual}, "
            f"debería ser el original: {stock_original}"
        )

```

test_anular_venta_restaura_stock_al_valor_original.

```

def test_anular_venta_restaura_stock_al_valor_original(
    self, db, admin, sucursal, producto
):
    """
    US-04: Después de registrar una venta y luego anularla,
    el stock debe quedar exactamente igual que al principio.
    """
    stock_original = producto.stock_actual

    resultado = ControladorVentas(db).registrar_venta(
        {"items": [{"producto_id": producto.id, "cantidad": 5}]},
        admin.id,
    )
    ControladorVentas(db).anular_venta(
        resultado["venta_id"], admin.id, "Error en digitación"
    )
    db.refresh(producto)

    assert producto.stock_actual == stock_original, (
        f"Stock después de anular: {producto.stock_actual}, "
        f"debería ser el original: {stock_original}"
    )

```

El caso captura el valor de stock_actual antes de la venta (stock_original), registra una venta de cinco unidades, la anula con un motivo válido y luego recarga el objeto Producto desde la base de datos para comparar su stock_actual contra stock_original. La condición de éxito es la igualdad exacta: ni una unidad más ni una menos.

Este test es especialmente relevante desde la perspectiva del negocio porque una anulación incorrecta generaría un inventario inflado que no correspondería a la mercancía física disponible, causando errores en reportes y en futuras ventas. La Tabla 4 presenta la ficha del caso.

Atributo	Detalle
Identificador	test_anular_venta_restaura_stock_al_valor_original
Funciones ejercitadas	registrar_venta() → anular_venta()
Tipo de validación	Consistencia transaccional, reversión de estado
Caso de negocio	Evitar inventario inflado o negativo tras anulación
Condición de éxito	stock_actual == stock_original (igualdad exacta)
Resultado obtenido	PASA

9.3.3.4 TestAlertasTrasMovimiento

Esta clase verifica la integración entre el módulo de movimientos de inventario y el módulo de alertas de stock, comprobando que una escritura en la base de datos se refleje inmediatamente en la consulta de alertas.

```

class TestAlertasTrasMovimiento:
    """
    Verifica que después de un movimiento de salida que deja el stock
    por debajo del mínimo, la función de alertas consulta la BD y
    retorna ese producto como urgente de reabastecer.
    """

    def test_salida_que_supera_minimo_activa_alerta(self, db, admin):
        # Crear producto justo sobre el mínimo
        producto = modelos.Producto(
            codigo="ALT01",
            nombre="Café Sello Rojo 500g",
            categoria="Bebidas",
            precio_venta=12500,
            costo=8500,
            stock_actual=11,
            stock_minimo=10,
            activo=True,
        )
        db.add(producto)
        db.commit()
        db.refresh(producto)

        # Registrar salida que lo deja bajo el mínimo
        ControladorInventario(db).registrar_movimiento(
            {
                "producto_id": producto.id,
                "tipo_movimiento": "salida",
                "cantidad": 2,
                "motivo": "Venta en caja",
            },
            admin.id,
        )

        alertas = ControladorInventario(db).verificar_alertas_inventario()
        ids_en_alerta = [a["producto_id"] for a in alertas]

        assert producto.id in ids_en_alerta, (
            "El producto bajó del mínimo pero no apareció en las alertas"
        )

```

test_salida_que_supera_minimo_activa_alerta.

```

def test_salida_que_supera_minimo_activa_alerta(self, db, admin):
    # Crear producto justo sobre el mínimo
    producto = modelos.Producto(
        codigo="ALT01",
        nombre="Café Sello Rojo 500g",
        categoria="Bebidas",
        precio_venta=12500,
        costo=8500,
        stock_actual=11,
        stock_minimo=10,
        activo=True,
    )
    db.add(producto)
    db.commit()
    db.refresh(producto)

    # Registrar salida que lo deja bajo el mínimo
    ControladorInventario(db).registrar_movimiento(
        {
            "producto_id": producto.id,
            "tipo_movimiento": "salida",
            "cantidad": 2,
            "motivo": "Venta en caja",
        },
        admin.id,
    )

    alertas = ControladorInventario(db).verificar_alertas_inventario()
    ids_en_alerta = [a["producto_id"] for a in alertas]

    assert producto.id in ids_en_alerta, (
        "El producto bajó del mínimo pero no apareció en las alertas"
    )

```

El caso crea un producto con `stock_actual=11` y `stock_minimo=10`, es decir, un producto ubicado exactamente una unidad sobre el umbral. A continuación registra una salida de dos unidades, dejando el `stock_actual` en nueve, por debajo del mínimo configurado. Por último consulta las alertas y verifica que el identificador del producto aparece en la lista retornada.

Este test ejercita un valor límite: el cruce del umbral mínimo. Es la prueba que distingue si el sistema detecta proactivamente los productos en riesgo de agotamiento. Sin esta validación, podrían existir productos críticos que no generan alerta y provocan quiebres de stock que afectan directamente la operación de la tienda. La Tabla 5 presenta la ficha del caso.

Atributo	Detalle
Identificador	test_salida_que_supera_minimo_activa_alerta
Funciones ejercitadas	registrar_movimiento() → verificar_alertas_inventario()
Tipo de validación	Integración escritura-lectura y valor límite
Estado inicial	stock=11, mínimo=10 (una unidad sobre el umbral)
Acción aplicada	Salida de 2 unidades → stock queda en 9
Condición de éxito	producto.id aparece en la lista de alertas
Resultado obtenido	PASA

9.3.3.5 TestControlDeAccesoHTTP

Esta clase evalúa el control de acceso a nivel de API HTTP. A diferencia de las clases anteriores, que llamaban directamente a los controladores, estos tests envían peticiones HTTP reales a través del cliente TestClient y verifican los códigos de respuesta. Es la clase de prueba más cercana al nivel de sistema dentro del conjunto de integración.

```

class TestControlDeAccesoHTTP:
    """
    Verifica que las restricciones de rol funcionan a nivel de API,
    no solo en la interfaz visual.
    """

    def test_petition_sin_sesion_retorna_401(self, client):
        respuesta = client.post("/api/ventas", json={"items": []})

        assert respuesta.status_code == 401, (
            f"Se esperaba 401 (no autenticado), se recibió: {respuesta.status_code}"
        )

    def test_cajero_no_puede_anular_ventas(self, client, sesion_cajero, producto):
        """
        Un cajero autenticado que intenta anular una venta debe
        recibir HTTP 403 (prohibido), sin importar que la venta exista.
        """
        # El cajero registra la venta (esto sí puede hacerlo)
        respuesta_venta = client.post(
            "/api/ventas",
            json={"items": [{"producto_id": producto.id, "cantidad": 1}]},
            headers=sesion_cajero,
        )
        venta_id = respuesta_venta.json()["venta_id"]

        # El cajero intenta anularla (esto NO puede hacerlo)
        respuesta_anulacion = client.patch(
            f"/api/ventas/{venta_id}",
            json={"motivo": "Intento no autorizado"},
            headers=sesion_cajero,
        )

        assert respuesta_anulacion.status_code == 403, (
            "El cajero pudo anular una venta cuando no debería tener ese permiso"
        )

```

`test_petition_sin_sesion_retorna_401.`

```

def test_petition_sin_sesion_retorna_401(self, client):
    respuesta = client.post("/api/ventas", json={"items": []})

    assert respuesta.status_code == 401, (
        f"Se esperaba 401 (no autenticado), se recibió: {respuesta.status_code}"
    )

```

Este caso envía una petición de creación de venta sin incluir el header session-id y verifica que la respuesta sea HTTP 401 (No autorizado). Valida que la autenticación es obligatoria y que ningún endpoint sensible es accesible de forma pública. El riesgo de no contar con esta validación es que un endpoint pudiera quedar expuesto, permitiendo registros de ventas sin ninguna identificación del actor.

`test_cajero_no_puede_anular_ventas.`


```

def test_cajero_no_puede_anular_ventas(self, client, sesion_cajero, producto):
    """
    Un cajero autenticado que intenta anular una venta debe
    recibir HTTP 403 (prohibido), sin importar que la venta exista.
    """
    # El cajero registra la venta (esto sí puede hacerlo)
    respuesta_venta = client.post(
        "/api/ventas",
        json={"items": [{"producto_id": producto.id, "cantidad": 1}]},
        headers=sesion_cajero,
    )
    venta_id = respuesta_venta.json()["venta_id"]

    # El cajero intenta anularla (esto NO puede hacerlo)
    respuesta_anulacion = client.patch(
        f"/api/ventas/{venta_id}",
        json={"motivo": "Intento no autorizado"},
        headers=sesion_cajero,
    )

    assert respuesta_anulacion.status_code == 403, (
        "El cajero pudo anular una venta cuando no debería tener ese permiso"
    )

```

Este caso primero registra una venta utilizando la sesión del cajero, una operación que ese rol tiene permitida. Luego intenta anular esa misma venta con el mismo header de cajero. El resultado esperado es HTTP 403 (Prohibido). Esta prueba valida el control de acceso basado en roles (RBAC): la anulación de ventas es una operación exclusiva de la administradora, y el sistema debe rechazar el intento del cajero independientemente de que este esté autenticado correctamente.

El riesgo de no contar con esta prueba es grave desde el punto de vista del negocio: un cajero podría anular sus propias ventas para encubrir diferencias de caja o cometer fraude sin que el sistema lo impidiera. La Tabla 6 compara ambos casos de la clase.

Atributo	Test 1	Test 2
Endpoint	POST /api/ventas	PATCH /api/ventas/{id}
Actor	Sin autenticación	Cajero autenticado

Tipo de validación	Autenticación obligatoria	Autorización por rol (RBAC)
Respuesta esperada	HTTP 401	HTTP 403
Riesgo si falla	Endpoint expuesto públicamente	Cajero puede anular ventas (fraude)
Resultado	PASA	PASA

Nota. Comparativa de los dos casos de la clase TestControlDeAccesoHTTP.

9.3.4 Casos de Frontera Documentados

Los casos de frontera son escenarios ubicados en los límites de las particiones de equivalencia y que con mayor frecuencia revelan defectos de implementación. Las pruebas de integración de StoreVision cubren cinco casos de frontera:

- **Transacciones con ítems mixtos.** Una venta compuesta por un ítem válido y uno inválido debe activar el rollback completo, verificando que el sistema no persiste estados intermedios.
- **Restauración exacta de stock.** La anulación de una venta debe devolver el stock en la misma cantidad descontada, sin diferencia de ± 1 unidad, verificando la aritmética de la reversión.
- **Cruce del umbral mínimo.** Un producto con stock_actual=11 y stock_minimo=10 que recibe una salida de dos unidades queda en nueve, verificando que la alerta se activa exactamente al cruzar el umbral.
- **Acceso sin autenticación.** Una petición a un endpoint protegido sin header de sesión debe producir HTTP 401, validando que la identificación del actor es obligatoria.
- **Acceso con rol incorrecto.** Un usuario autenticado con rol insuficiente debe recibir HTTP 403, validando que autenticación y autorización son mecanismos independientes.

9.3.5 Resultados de Ejecución

Los siete casos del archivo `test_pi_integracion.py` fueron ejecutados con el comando `pytest tests/test_pi_integracion.py -v` en un entorno Windows con Python 3.12.10, pytest 9.0.3 y base de datos SQLite en memoria. El tiempo total de ejecución fue de 1.99 segundos. La Tabla 7 presenta el resumen completo de resultados.

```
(.venv) PS C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III> cd prototipos
(.venv) PS C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\prototipos> cd prototipo1
(.venv) PS C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\prototipos\prototipo1> pytest tests/test_pi_integracion.py -v

===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\prototipos\prototipo1
configfile: pytest.ini
plugins: anyio-4.12.1
collected 7 items

tests/test_pi_integracion.py::TestAuditoriaDeAcceso::test_login_exitoso_queda_en_tabla_auditoria PASSED [ 14%]
tests/test_pi_integracion.py::TestAtomicidadDeVentas::test_venta_exitosa_descuenta_stock_y_crea_movimiento PASSED [ 28%]
tests/test_pi_integracion.py::TestAtomicidadDeVentas::test_venta_con_producto_inexistente_no_deja_registros_parciales PASSED [ 42%]
tests/test_pi_integracion.py::TestAnulacionRestaurandoStock::test_anular_venta_restaura_stock_al_valor_original PASSED [ 57%]
tests/test_pi_integracion.py::TestAlertasTrasMovimiento::test_salida_que_supera_minimo_activa_alerta PASSED [ 71%]
tests/test_pi_integracion.py::TestControlDeAccesoHTTP::test_peticion_sin_sesion_retorna_401 PASSED [ 85%]
tests/test_pi_integracion.py::TestControlDeAccesoHTTP::test_cajero_no_puede_anular_ventas PASSED [100%]

===== warnings summary =====
models/database.py:11
  C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\prototipos\prototipo1\models\database.py:11: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()

tests/test_pi_integracion.py::TestControlDeAccesoHTTP::test_cajero_no_puede_anular_ventas
  C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\prototipos\prototipo1\views\api_views.py:58: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    session_id = f"session_{usuario.id}_{datetime.utcnow().timestamp()}"

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 7 passed, 2 warnings in 1.99s =====
(.venv) PS C:\Users\Usuario\Downloads\Proyectos\ING_SOFTWARE_III\prototipos\prototipo1>
```

Nota. Resultados completos de ejecución de `test_pi_integracion.py`. Entorno: Python 3.12.10, pytest 9.0.3, Windows 11.

Los resultados confirman que los siete casos de integración pasan satisfactoriamente, validando la correcta interacción entre los controladores de autenticación, ventas e inventario, la base de datos y los endpoints HTTP de la API REST de StoreVision.

9.4 Implementación Pruebas de Sistema

9.4.1 Definición de Prueba de Sistema

Las pruebas de sistema verifican el comportamiento del software desde la perspectiva del usuario final, evaluando flujos completos de extremo a extremo a través de la interfaz HTTP de la aplicación. A diferencia de las pruebas de integración, que conocen parcialmente la estructura interna del sistema, las pruebas de sistema actúan como caja negra: el evaluador no tiene conocimiento del código, los controladores ni el esquema de la base de datos;

únicamente interactúa con los endpoints expuestos y evalúa los códigos de respuesta y los datos retornados.

En StoreVision, este nivel de prueba verifica que el flujo completo del cajero —seleccionar productos, confirmar la venta y observar el descuento de stock— funcione correctamente desde el primer request HTTP hasta el último efecto en la base de datos, exactamente como lo haría un usuario real en producción.

9.4. 2 Justificación del Uso de Caja Negra

La técnica de caja negra trata al sistema como un componente opaco: el evaluador define las entradas, ejecuta las acciones y verifica las salidas sin ningún acceso al código fuente. Esta técnica es la más adecuada para el nivel de sistema por tres razones:

Simula el comportamiento real del usuario. Las pruebas actúan exactamente como lo haría el cajero o la administradora en el punto de venta: enviando peticiones HTTP y evaluando las respuestas, sin atajos hacia la lógica interna.

Detecta defectos de integración no visibles en niveles inferiores. Errores en la configuración de rutas o transformaciones de respuesta solo se manifiestan cuando el sistema se evalúa en su totalidad.

Valida el contrato de la API. Los códigos de estado HTTP (201, 200, 401, 403) y la estructura del cuerpo JSON son el contrato entre el backend y cualquier cliente; la caja negra garantiza que ese contrato se cumpla.

La diferencia fundamental entre los tres niveles de prueba aplicados en StoreVision se resume en la siguiente tabla

Comparativa entre los Tres Niveles de Prueba Aplicados en StoreVision

Característica	Unitaria	Integración	Sistema
Técnica	Caja blanca	Caja gris	Caja negra
Alcance	Función aislada	Capas	Flujo HTTP

		interactuando	completo
BD real	No	Sí (en memoria)	Sí (en memoria)
Conocimiento interno	Total	Parcial	Ninguno

Fixtures Utilizadas en las Pruebas de Sistema

Las fixtures están definidas en `confest.py` y proveen el contexto reutilizable que todos los tests comparten. En las pruebas de sistema, las fixtures levantan el cliente HTTP completo y crean sesiones de usuario autenticadas a través del mismo flujo de login que usaría un usuario real.

Base de datos en memoria.

La fixture `db` crea un motor SQLite con `StaticPool` que garantiza que todas las operaciones de una prueba compartan la misma conexión. Antes de cada test se crean todas las tablas y al finalizar se destruyen, produciendo aislamiento total entre casos. SQLite en memoria replica el comportamiento transaccional de una base de datos relacional sin la latencia de disco.

Cliente HTTP.

La fixture `client` crea una instancia de `TestClient` de FastAPI que apunta a la aplicación completa de producción. Esto permite enviar peticiones HTTP reales y obtener respuestas completas sin levantar un servidor Uvicorn, evaluando el sistema exactamente como lo haría un cliente externo.

Usuarios y sesiones autenticadas.

Las fixtures `sesion_admin` y `sesion_cajero` realizan un login real vía `POST /api/login` y capturan el `session-id` retornado, que luego se incluye como header en cada petición. Desde la perspectiva de caja negra, esto equivale a que el evaluador abre el sistema, introduce sus credenciales y opera con la sesión activa que le entrega la aplicación.

Producto y fixture de límite.

La fixture producto provee un Producto con stock_actual=100 y stock_minimo=10, valores que permiten ejecutar ventas sin interferir con las alertas de inventario. La fixture producto_en_limite provee un producto configurado con stock en el umbral mínimo para verificar el comportamiento del endpoint de alertas.

9.4.3 Análisis Detallado de Cada Clase de Prueba

9.4.3.1 TestFlujoDeVenta.

```
class TestFlujoDeVenta:
    """
    Simula el flujo real del cajero en caja: selecciona productos,
    confirma la venta y el sistema descuenta el stock automáticamente.
    """
```

Esta clase simula el flujo real del cajero en caja: selecciona un producto disponible, confirma la venta y verifica que el sistema responde correctamente y descuenta el stock de forma automática. Es la clase central del conjunto de pruebas de sistema porque ejerce el caso de uso principal del negocio.

test_venta_exitosa_retorna_201.

```
def test_venta_exitosa_retorna_201(self, client, sesion_cajero, producto):
    """
    Una venta con producto disponible debe retornar HTTP 201
    (recurso creado exitosamente).
    """
    respuesta = client.post(
        "/api/ventas",
        json={"items": [{"producto_id": producto.id, "cantidad": 2}]},
        headers=sesion_cajero,
    )

    assert respuesta.status_code == 201, (
        f"Se esperaba 201, se recibió {respuesta.status_code}. "
        f"Detalle: {respuesta.json()}"
    )
```

Este caso envía una petición POST a /api/ventas con un ítem de dos unidades de un producto con stock suficiente y verifica que la respuesta sea HTTP 201. Desde la perspectiva de caja negra, el evaluador únicamente sabe que registrar una venta exitosa debe producir un recurso creado; no tiene acceso a los controladores ni a la base de datos. El código 201 es el contrato que el sistema ofrece a sus clientes.

`test_venta_descuenta_stock_en_base_de_datos.`

```
def test_venta_descuenta_stock_en_base_de_datos(
    self, client, db, sesion_cajero, producto
):
    """
    Después de confirmar la venta, el stock en la BD debe
    reflejar las unidades vendidas, sin necesidad de ninguna acción manual.
    """
    stock_antes = producto.stock_actual
    client.post(
        "/api/ventas",
        json={"items": [{"producto_id": producto.id, "cantidad": 3}]},
        headers=sesion_cajero,
    )
    db.refresh(producto)

    assert producto.stock_actual == stock_antes - 3, (
        f"Stock esperado: {stock_antes - 3}, actual: {producto.stock_actual}"
    )
```

Este caso verifica que, después de confirmar una venta de tres unidades, el stock del producto en la base de datos se reduzca exactamente en tres. Aunque la validación accede a la

base de datos para verificar el efecto, la acción se realiza exclusivamente a través del endpoint HTTP, sin llamar directamente a ningún controlador. Este enfoque garantiza que el flujo completo —request, procesamiento, persistencia— funcione de extremo a extremo.

El riesgo de no contar con esta prueba es que el sistema podría registrar la venta y retornar 201 sin descontar el stock, generando un inventario lógico que no refleja la mercancía vendida y causando diferencias de caja indetectables de forma automática. Las siguientes tablas presentan la ficha del caso principal y la comparación con ambos casos de la clase.

Ficha del Caso test_venta_exitosa_retorna_201

Atributo	Detalle
Identificador	test_venta_exitosa_retorna_201
Clase de prueba	TestFlujoDeVenta
Endpoint ejercitado	POST /api/ventas
Tipo de validación	Funcional — código de estado HTTP
Partición de equivalencia	Producto disponible en stock (partición positiva)
Resultado esperado	HTTP 201 (recurso creado exitosamente)
Resultado obtenido	PASA

Nota. Ficha del caso principal de la clase TestFlujoDeVenta.

Comparativa de los Dos Casos de la Clase TestFlujoDeVenta

Atributo	Test 1	Test 2
Escenario	Venta con stock suficiente	Venta con descuento en BD

Tipo de validación	Código HTTP 201	Persistencia en base de datos
Verificación	status_code == 201	stock_actual == stock_antes - 3
Resultado	PASA	PASA

Nota. Ambos casos ejercitan el mismo endpoint POST /api/ventas con validaciones complementarias.

9.4.3.2 *TestAnulacionDeVentas.*

```
class TestAnulacionDeVentas:
    """
    Verifica el flujo de anulación y que los permisos funcionan
    correctamente a nivel HTTP.
    """
```

Esta clase verifica el flujo de anulación de ventas y garantiza que los permisos funcionan correctamente a nivel HTTP. Contiene dos casos complementarios que validan el mismo endpoint con actores distintos, produciendo resultados opuestos en función del rol.

test_administradora_puede_anular_venta.

```
def test_administradora_puede_anular_venta(
    self, client, sesion_admin, sesion_cajero, producto
):
    # El cajero registra la venta
    respuesta_venta = client.post(
        "/api/ventas",
        json={"items": [{"producto_id": producto.id, "cantidad": 1}]},
        headers=sesion_cajero,
    )
    venta_id = respuesta_venta.json()["venta_id"]

    # La administradora la anula
    respuesta_anulacion = client.patch(
        f"/api/ventas/{venta_id}",
        json={"motivo": "Error en digitación del cajero"},
        headers=sesion_admin,
    )

    assert respuesta_anulacion.status_code == 200, (
        f"La administradora no pudo anular la venta. "
        f"Código: {respuesta_anulacion.status_code}"
    )
```

Este caso ejecuta un flujo de dos pasos: primero el cajero registra una venta vía POST `/api/ventas` y captura el `venta_id` retornado; luego la administradora anula esa venta vía PATCH `/api/ventas/{venta_id}` con un motivo válido. El resultado esperado es HTTP 200. La prueba valida que el sistema respeta el control de acceso basado en roles permitiendo la operación al rol con los permisos correctos.

`test_cajero_no_puede_anular_ventas.`

```
def test_cajero_no_puede_anular_ventas(self, client, sesion_cajero, producto):
    respuesta_venta = client.post(
        "/api/ventas",
        json={"items": [{"producto_id": producto.id, "cantidad": 1}]},
        headers=sesion_cajero,
    )
    venta_id = respuesta_venta.json()["venta_id"]

    respuesta_anulacion = client.patch(
        f"/api/ventas/{venta_id}",
        json={"motivo": "Intento no autorizado"},
        headers=sesion_cajero,
    )

    assert respuesta_anulacion.status_code == 403, (
        "El cajero pudo anular una venta, lo cual representa un riesgo de fraude"
    )
```

Este caso replica el mismo flujo de dos pasos, pero utiliza la sesión del cajero para ambas operaciones. El resultado esperado es HTTP 403 (Prohibido). Desde la perspectiva de caja negra, esto verifica que autenticación y autorización son mecanismos independientes: el cajero está correctamente autenticado, pero no tiene autorización para esa operación.

El riesgo de no contar con esta prueba es grave desde el punto de vista del negocio: un cajero podría anular sus propias ventas para encubrir diferencias de caja o cometer fraude sin que el sistema lo impidiera. La siguiente tabla compara ambos casos de la clase.

Comparativa de los Dos Casos de la Clase TestAnulacionDeVentas

Atributo	Test 1	Test 2
Escenario	Administradora anula venta	Cajero intenta anular venta
Endpoint	PATCH /api/ventas/{id}	PATCH /api/ventas/{id}
Actor	sesion_admin	sesion_cajero
Tipo de validación	Autorización por rol (RBAC)	Autorización por rol (RBAC)
Respuesta esperada	HTTP 200	HTTP 403
Riesgo si falla	Ventas no se pueden anular	Cajero puede cometer fraude
Resultado	PASA	PASA

Nota. Los dos casos validan el mismo endpoint con actores distintos, cubriendo el acceso legítimo y el intento no autorizado.

9.4.3.3 TestAlertasDeInventario.

```
class TestAlertasDeInventario:
    """
    Verifica que el endpoint de alertas retorna los productos correctos.
    """
```

Esta clase verifica que el endpoint de alertas de inventario retorna exactamente los productos correctos: ni omite productos que deben alertarse ni genera falsas alarmas para productos con stock normal.

test_producto_bajo_minimo_aparece_en_alertas.

```
def test_producto_bajo_minimo_aparece_en_alertas(self, client, producto_en_limite):
    """
    Un producto con stock en el límite mínimo debe aparecer
    en el listado de alertas del endpoint /api/inventario/alertas.
    """

    respuesta = client.get("/api/inventario/alertas")

    assert respuesta.status_code == 200
    ids_en_alerta = [a["producto_id"] for a in respuesta.json()]
    assert producto_en_limite.id in ids_en_alerta, (
        "El producto bajo mínimo no aparece en las alertas del sistema"
    )
```

Este caso utiliza la fixture `producto_en_limite`, que provee un producto con stock en el umbral mínimo, y consulta GET `/api/inventario/alertas`. Verifica que el identificador del producto aparece en la lista retornada. La prueba valida que el sistema detecta proactivamente los productos en riesgo de agotamiento.

El riesgo de no contar con esta prueba es que productos críticos podrían llegar a agotarse sin que el sistema lo señale, causando quiebres de stock que afectan directamente la operación de la tienda.

test_producto_con_stock_normal_no_genera_alerta_falsa.

```
def test_producto_con_stock_normal_no_genera_alerta_falsa(self, client, producto):
    ids_en_alerta = [
        a["producto_id"] for a in client.get("/api/inventario/alertas").json()
    ]

    assert producto.id not in ids_en_alerta, (
        "Un producto con stock normal apareció en alertas (falsa alarma)"
    )
```

Este caso utiliza la fixture `producto estándar`, con `stock_actual=100` y `stock_minimo=10`, y verifica que su identificador NO aparece en el listado de alertas. Esta

prueba de valor negativo es igualmente importante: un sistema que genera alertas falsas produce ruido que lleva a los operadores a ignorar las alertas reales. La siguiente tabla presenta la ficha del caso principal de la clase.

Ficha del Caso test_producto_bajo_minimo_aparece_en_alertas

Atributo	Detalle
Identificador	test_producto_bajo_minimo_aparece_en_alertas
Clase de prueba	TestAlertasDeInventario
Endpoint ejercitado	GET /api/inventario/alertas
Tipo de validación	Funcional — contenido de respuesta
Fixture utilizada	producto_en_limite (stock en umbral mínimo)
Condición de éxito	producto_en_limite.id en ids_en_alerta
Resultado obtenido	PASA

Nota. Ficha del caso principal de la clase TestAlertasDeInventario.

9.4.3.4 TestControlDeRoles.

```
class TestControlDeRoles:
    """
    Tabla de decisión de acceso a la API:
    Sin sesión          → 401 en cualquier operación
    Cajero, crea prod   → 403 (no tiene ese permiso)
    Admin, crea prod    → 201 (tiene acceso completo)
    """
```

Esta clase aplica una tabla de decisión de acceso a la API que cubre las tres condiciones posibles: sin sesión activa, sesión de cajero y sesión de administradora. Es la clase que valida de forma más directa el modelo de seguridad del sistema.

test_sin_sesion_retorna_401.

```
def test_sin_sesion_retorna_401(self, client):
    """
    Cualquier operación sin autenticación debe ser rechazada.
    """
    respuesta = client.post("/api/ventas", json={"items": []})

    assert respuesta.status_code == 401
```

Este caso envía una petición POST /api/ventas sin ningún header de autenticación y verifica que la respuesta sea HTTP 401 (No autorizado). Valida que ningún endpoint sensible es accesible de forma pública. El riesgo de no contar con esta validación es que un endpoint pudiera quedar expuesto, permitiendo registros de ventas sin identificación del actor.

test_cajero_no_puede_crear_productos.

```
def test_cajero_no_puede_crear_productos(self, client, sesion_cajero):
    """
    Crear productos es una función exclusiva de la
    administradora. El cajero debe recibir HTTP 403.
    """
    respuesta = client.post(
        "/api/productos",
        json={
            "codigo": "PROD_TEST",
            "nombre": "Producto de prueba",
            "categoria": "Test",
            "precio_venta": 1000,
            "costo": 500,
        },
        headers=sesion_cajero,
    )

    assert respuesta.status_code == 403, (
        "El cajero pudo crear un producto cuando no debería tener ese permiso"
    )
```

Este caso envía un POST /api/productos con la sesión del cajero y verifica HTTP 403. Crear y modificar el catálogo de productos es una función exclusiva de la administradora. El riesgo es que un cajero pudiera alterar precios o agregar productos ficticios para encubrir manipulaciones.

test_administradora_puede_crear_productos.

```
def test_administradora_puede_crear_productos(self, client, sesion_admin):
    """
    La administradora tiene acceso completo y puede crear
    nuevos productos en el catálogo.
    """
    respuesta = client.post(
        "/api/productos",
        json={
            "codigo": "NUEV099",
            "nombre": "Producto Nuevo",
            "categoria": "Test",
            "precio_venta": 1000,
            "costo": 500,
            "stock_minimo": 5,
        },
        headers=sesion_admin,
    )

    assert respuesta.status_code == 201, (
        f"La administradora no pudo crear un producto. "
        f"Código: {respuesta.status_code}, detalle: {respuesta.json()}"
    )
```

Este caso envía el mismo POST /api/productos con la sesión de la administradora y verifica HTTP 201. Valida que la administradora tiene acceso completo y que el sistema permite las operaciones legítimas. La siguiente tabla presenta la tabla de decisión de la clase.

Tabla de Decisión de Acceso de la Clase TestControlDeRoles

Atributo	Test 1	Test 2	Test 3
Escenario	Sin sesión activa	Cajero crea producto	Admin crea producto
Endpoint	POST /api/ventas	POST /api/productos	POST /api/productos
Actor	Sin autenticación	sesion_cajero	sesion_admin
Tipo de validación	Autenticación obligatoria	Autorización RBAC	Acceso completo
Respuesta esperada	HTTP 401	HTTP 403	HTTP 201

Riesgo si falla	API expuesta públicamente	Cajero altera el catálogo	Admin no puede gestionar
Resultado	PASA	PASA	PASA

Nota. Los tres casos cubren todas las combinaciones de autenticación y autorización relevantes para el control de acceso de la API.

9.4.4 Casos de Frontera Documentados

Los casos de frontera son escenarios ubicados en los límites de las particiones de equivalencia y que con mayor frecuencia revelan defectos de implementación. Las pruebas de sistema de StoreVision cubren los siguientes casos de frontera:

Respuesta exacta al crear una venta exitosa. El sistema debe retornar HTTP 201 y no otro código, verificando la precisión del código de estado en el contrato de la API.

Descuento exacto de stock. La venta de tres unidades debe reducir el stock exactamente en tres, verificando la aritmética de persistencia sin diferencias de ± 1 unidad.

Rol insuficiente en operaciones críticas. Un cajero autenticado que intenta anular ventas o crear productos debe recibir HTTP 403, validando que autenticación y autorización son mecanismos independientes.

Acceso sin autenticación. Una petición sin header de sesión debe producir HTTP 401, validando que la identificación del actor es obligatoria en todos los endpoints sensibles.

Producto exactamente en el umbral mínimo. Un producto con stock en el límite configurado debe aparecer en las alertas, verificando que el umbral se evalúa de forma inclusiva.

Ausencia de falsas alarmas. Un producto con stock holgado no debe aparecer en el listado de alertas, validando la precisión del filtro del endpoint.

9.4.5 Resultados de Ejecución

Los nueve casos del archivo `test_ps_sistema.py` fueron ejecutados con el comando `pytest tests/test_ps_sistema.py -v` en un entorno Windows con Python 3.12.10, pytest 9.0.3 y base de datos SQLite en memoria. Los nueve casos pasaron satisfactoriamente. La siguiente tabla presenta el resumen completo de resultados.

```

(.venv) PS C:\Users\Del1\ING_SOFTWARE_III\Prototipos\Prototipo1> pytest tests/test_ps_sistema.py -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Del1\ING_SOFTWARE_III\Prototipos\Prototipo1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Del1\ING_SOFTWARE_III\Prototipos\Prototipo1
configfile: pytest.ini
plugins: anyio-4.13.0
collected 9 items

tests/test_ps_sistema.py::TestFlujoDeVenta::test_venta_exitosa_retorna_201 PASSED
[ 11%]
tests/test_ps_sistema.py::TestFlujoDeVenta::test_venta_descuenta_stock_en_base_de_datos PASSED
[ 22%]
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_administradora_puede_anular_venta PASSED
[ 33%]
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_cajero_no_puede_anular_ventas PASSED
[ 44%]
tests/test_ps_sistema.py::TestAlertasDeInventario::test_producto_bajo_minimo_aparece_en_alertas PASSED
[ 55%]
tests/test_ps_sistema.py::TestAlertasDeInventario::test_producto_con_stock_normal_no_genera_alerta_falsa PASSED
[ 66%]
tests/test_ps_sistema.py::TestControlDeRoles::test_sin_sesion_retorna_401 PASSED
[ 77%]
tests/test_ps_sistema.py::TestControlDeRoles::test_cajero_no_puede_crear_productos PASSED
[ 88%]
tests/test_ps_sistema.py::TestControlDeRoles::test_administradora_puede_crear_productos PASSED
[100%]

===== warnings summary =====
models/database.py:11
  C:\Users\Del1\ING_SOFTWARE_III\Prototipos\Prototipo1\models\database.py:11: MovedIn20Warning: The ``declarative_base()`` function is now available as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/20)
    Base = declarative_base()

tests/test_ps_sistema.py::TestFlujoDeVenta::test_venta_exitosa_retorna_201
tests/test_ps_sistema.py::TestFlujoDeVenta::test_venta_descuenta_stock_en_base_de_datos
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_administradora_puede_anular_venta
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_administradora_puede_anular_venta
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_cajero_no_puede_anular_ventas
tests/test_ps_sistema.py::TestAlertasDeInventario::test_producto_bajo_minimo_aparece_en_alertas
tests/test_ps_sistema.py::TestAlertasDeInventario::test_producto_con_stock_normal_no_genera_alerta_falsa
tests/test_ps_sistema.py::TestControlDeRoles::test_cajero_no_puede_crear_productos

```

Resumen de Resultados de Ejecución de `test_ps_sistema.py`

Caso de prueba	Clase	Resultado
test_venta_exitosa_retorna_201	TestFlujoDeVenta	PASSED
test_venta_descuenta_stock_en_base_de_datos	TestFlujoDeVenta	PASSED
test_administradora_puede_anular_venta	TestAnulacionDe Ventas	PASSED
test_cajero_no_puede_anular_ventas	TestAnulacionDe Ventas	PASSED
test_producto_bajo_minimo_aparece_en_alertas	TestAlertasDeInve ntario	PASSED
test_producto_con_stock_normal_no_genera_alerta_fa	TestAlertasDeInve	PASSED

lsa	ntario	
test_sin_sesion_retorna_401	TestControlDeRoles	PASSED
test_cajero_no_puede_crear_productos	TestControlDeRoles	PASSED
test_administradora_puede_crear_productos	TestControlDeRoles	PASSED

Nota. Entorno: Python 3.12.10, pytest 9.0.3, Windows 11. La advertencia de deprecación de SQLAlchemy 2.0 (declarative_base) no afecta los resultados.

9.5 Conclusión General de Pruebas

```

===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- /home/david/Documents/Universidad/QuintoSemestre/Software III/proyectoStoreVision/ING_SOFTWARE_III/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/david/Documents/Universidad/QuintoSemestre/Software III/proyectoStoreVision/ING_SOFTWARE_III/Prototipos/Prototipo1
configfile: pytest.ini
plugins: anyio-4.12.1
collected 32 items

tests/test_pi_integracion.py::TestAuditoriaDeAcceso::test_login_exitoso_queda_en_tabla_auditoria PASSED [ 3%]
tests/test_pi_integracion.py::TestAtomicidadDeVentas::test_venta_exitosa_descuenta_stock_y_crea_movimiento PASSED [ 6%]
tests/test_pi_integracion.py::TestAtomicidadDeVentas::test_venta_con_producto_inexistente_no_deja_registros_parciales PASSED [ 9%]
tests/test_pi_integracion.py::TestAnulacionReintegrandoStock::test_anular_venta_restaura_stock_al_valor_original PASSED [ 12%]
tests/test_pi_integracion.py::TestAlertasTrasMovimiento::test_salida_que_supera_minimo_activa_alerta PASSED [ 15%]
tests/test_pi_integracion.py::TestControlDeAccesoHTTP::test_petición_sin_sesion_retorna_401 PASSED [ 18%]
tests/test_pi_integracion.py::TestControlDeAccesoHTTP::test_cajero_no_puede_anular_ventas PASSED [ 21%]
tests/test_ps_sistema.py::TestFlujoDeVenta::test_venta_exitosa_retorna_201 PASSED [ 25%]
tests/test_ps_sistema.py::TestFlujoDeVenta::test_venta_descuenta_stock_en_base_de_datos PASSED [ 28%]
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_administradora_puede_anular_venta PASSED [ 31%]
tests/test_ps_sistema.py::TestAnulacionDeVentas::test_cajero_no_puede_anular_ventas PASSED [ 34%]
tests/test_ps_sistema.py::TestAlertasDeInventario::test_producto_bajo_minimo_aparece_en_alertas PASSED [ 37%]
tests/test_ps_sistema.py::TestAlertasDeInventario::test_producto_con_stock_normal_no_genera_alerta_falsa PASSED [ 40%]
tests/test_ps_sistema.py::TestControlDeRoles::test_sin_sesion_retorna_401 PASSED [ 43%]
tests/test_ps_sistema.py::TestControlDeRoles::test_cajero_no_puede_crear_productos PASSED [ 46%]
tests/test_ps_sistema.py::TestControlDeRoles::test_administradora_puede_crear_productos PASSED [ 50%]
tests/test_pu_unitarias.py::TestAutenticacion::test_credenciales_correctas_devuelven_usuario PASSED [ 53%]
tests/test_pu_unitarias.py::TestAutenticacion::test_password_incorrecta_bloquea_acceso PASSED [ 56%]
tests/test_pu_unitarias.py::TestAutenticacion::test_login_fallido_queda_registrado_en_auditoria PASSED [ 59%]
tests/test_pu_unitarias.py::TestAutenticacion::test_email_duplicado_no_crea_usuario PASSED [ 62%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_valida_genera_id_de_venta PASSED [ 65%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_total_es_cantidad_por_precio_unitario PASSED [ 68%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_sin_productos_devuelve_error PASSED [ 71%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_con_stock_insuficiente_no_modifica_inventario PASSED [ 75%]
tests/test_pu_unitarias.py::TestRegistrarVenta::test_venta_con_producto_agotado_devuelve_error PASSED [ 78%]
tests/test_pu_unitarias.py::TestAnularVenta::test_no_se_puede_anular_venta_ya_anulada PASSED [ 81%]
tests/test_pu_unitarias.py::TestMovimientosInventario::test_entrada_de_mercancia_incrementa_el_stock PASSED [ 84%]
tests/test_pu_unitarias.py::TestMovimientosInventario::test_salida_mayor_al_stock_devuelve_error PASSED [ 87%]
tests/test_pu_unitarias.py::TestAlertasStock::test_producto_en_limite_minimo_genera_alerta PASSED [ 90%]
tests/test_pu_unitarias.py::TestAlertasStock::test_producto_con_stock_suficiente_no_genera_alerta PASSED [ 93%]
tests/test_pu_unitarias.py::TestAlertasStock::test_producto_agotado_tambien_genera_alerta PASSED [ 96%]
tests/test_pu_unitarias.py::TestBalanceEconomico::test_utilidad_bruta_es_ventas_menos_costos PASSED [100%]

```

Los resultados obtenidos tras la ejecución del conjunto de pruebas mediante pytest tests/ -v evidencian un comportamiento consistente y correcto del sistema en los escenarios evaluados. En total, se ejecutaron 32 casos de prueba distribuidos en los niveles unitario, de integración y de sistema, obteniendo un 100% de resultados exitosos (PASSED), lo que indica que las funcionalidades críticas implementadas cumplen con los criterios definidos en las historias de usuario priorizadas.

El éxito en las pruebas unitarias confirma que la lógica interna de los controladores se encuentra correctamente implementada, mientras que las pruebas de integración validan la adecuada interacción entre los distintos componentes del sistema, especialmente en procesos sensibles como el registro de ventas, la actualización de inventario y la gestión de transacciones. Por su parte, las pruebas de sistema demuestran que los flujos principales operan de manera coherente desde una perspectiva externa, cumpliendo con las respuestas esperadas a nivel de API.

No obstante, durante la ejecución se identificaron advertencias relacionadas con el uso de funciones deprecadas en bibliotecas como SQLAlchemy y manejo de fechas en Python, lo cual no afecta el resultado actual de las pruebas, pero representa un aspecto técnico que debe ser corregido en futuras iteraciones para garantizar la mantenibilidad y compatibilidad del sistema.

Como complemento al proceso de validación, se utilizó el comando `pytest tests/ --cov=. --cov-report=term-missing`, el cual permite ejecutar todas las pruebas del proyecto y, adicionalmente, medir la cobertura del código. En este contexto, la opción `--cov=.` indica que se analizará todo el código del proyecto, mientras que `--cov-report=term-missing` genera un reporte en la terminal mostrando qué líneas no han sido ejecutadas por las pruebas.

```

===== tests coverage =====
coverage: platform linux, python 3.14.4-final-0
Name                               Stmts  Miss  Cover   Missing
-----
controllers/_init_.py               0      0  100%
controllers/auth_controller.py      43     13    70%   33, 47-49, 59-84
controllers/inventario_controller.py 65     29    55%   17, 55-57, 77-78, 81-97, 100-123, 127-131
controllers/reportes_controller.py   63     41    35%   65-67, 70-121, 131-183
controllers/ventas_controller.py     89     27    70%   96-99, 105, 144-146, 149-161, 164-187, 190-194
main.py                             57     57     0%   1-132
models/_init_.py                    0      0  100%
models/database.py                  15      5    67%   14-18, 21
models/modelos.py                   75      0  100%
tests/conftest.py                   85      0  100%
tests/test_pi_integracion.py         53      0  100%
tests/test_ps_sistema.py            44      0  100%
tests/test_pu_unitarias.py          86      0  100%
views/_init_.py                     0      0  100%
views/api_views.py                  229    126    45%   38, 56, 79-81, 91-106, 111-114, 143, 170-172, 182-216, 225-253, 266-304, 309-310, 315-319, 350, 376, 382-383, 393-408, 424-454, 463-472, 486-494, 503-511, 520-528
TOTAL                               904    298    67%

===== 32 passed, 10 warnings in 16.09s =====

```

Los resultados obtenidos permiten evidenciar que la cobertura no es total, lo cual tiene sentido con el enfoque adoptado en esta etapa del proyecto. Se observa que los módulos asociados a funcionalidades críticas, como los controladores de ventas, autenticación e

inventario, presentan un nivel de cobertura mayor en comparación con otros componentes del sistema.

Por el contrario, módulos como reportes, vistas y el archivo principal presentan menor o nula cobertura, lo que confirma que no han sido priorizados en esta fase de pruebas. Este comportamiento valida que la estrategia implementada se ha enfocado deliberadamente en evaluar únicamente las partes más críticas del sistema, de acuerdo con las historias de usuario seleccionadas.

En este sentido, el análisis de cobertura no solo permite identificar áreas no probadas, sino también justificar el alcance actual de las pruebas, evidenciando que la validación realizada está alineada con los objetivos definidos para esta etapa del desarrollo.